

APKOS: Accurate and Privacy-Preserving Keyword Spotting for Intelligent Voice Assistant Systems

PEIJIA ZHENG*, School of Computer Science and Engineering, Sun Yat-Sen University, Guangzhou, China
JINGYI CHEN, School of Computer Science and Engineering, Sun Yat-Sen University, Guangzhou, China
ZHUOYUAN CHEN, School of Computer Science and Engineering, Sun Yat-Sen University, Guangzhou, China
HUIYU ZHOU, Guangxi Key Laboratory of Digital Infrastructure, Guangxi Zhuang Autonomous Region Information Center, Nanning, China
GUORUI XIE, Pengcheng Laboratory, Shenzhen, China
QING LI, Pengcheng Laboratory, Shenzhen, China
JIWU HUANG, Shenzhen MSU-BIT University, Shenzhen, China

Intelligent voice assistants and their supporting Cloud Voice Assistant (CVA) systems are becoming increasingly prevalent in AIoT environments. However, transmitting voice data to the cloud raises significant privacy concerns. Homomorphic encryption (HE) offers a promising solution by enabling secure computation over encrypted data without revealing the plaintext. Yet, its application introduces performance challenges for keyword spotting (KWS). Therefore, designing a privacy-preserving KWS scheme that ensures both security and high accuracy remains a significant challenge. In this paper, we propose a privacy-preserving and accurate keyword spotting scheme tailored for CVA systems. Our design is based on HE and achieves semantic security while maintaining high KWS performance. To optimize encrypted inference, we introduce a fusion layer that combines activation and pooling operations, reducing ciphertext-level computation. We also design a real-valued polynomial activation function tailored for audio-based tasks and employ a kernel expansion strategy to enhance representational capacity without increasing multiplicative depth. Extensive evaluations across multiple datasets demonstrate that our method achieves up to 7% improvement in accuracy and 10% gain in robustness compared to state-of-the-art privacy-preserving KWS methods.

CCS Concepts: • **Security and privacy** → **Privacy-preserving protocols**; **Usability in security and privacy**; • **Computing methodologies** → *Speech recognition*.

Additional Key Words and Phrases: Privacy-preserving inference, homomorphic encryption, keyword spotting, multimedia security

*This work is in part supported by Open Project Program of Guangxi Key Laboratory of Digital Infrastructure (Grant Number: GXDIOP2024015).

Authors' Contact Information: Peijia Zheng, School of Computer Science and Engineering, Sun Yat-Sen University, Guangzhou, Guangdong, China; e-mail: zhpj@mail.sysu.edu.cn; Jingyi Chen, School of Computer Science and Engineering, Sun Yat-Sen University, Guangzhou, Guangdong, China; e-mail: chenji563@mail2.sysu.edu.cn; Zhuoyuan Chen, School of Computer Science and Engineering, Sun Yat-Sen University, Guangzhou, Guangdong, China; e-mail: chenzhy285@mail2.sysu.edu.cn; Huiyu Zhou, Guangxi Key Laboratory of Digital Infrastructure, Guangxi Zhuang Autonomous Region Information Center, Nanning, Guangxi, China; e-mail: zhouhy3296@gmail.com; Guorui Xie, Pengcheng Laboratory, Shenzhen, Guangdong, China; e-mail: xiegr@gmail.com; Qing Li, Pengcheng Laboratory, Shenzhen, Guangdong, China; e-mail: andyliqing@gmail.com; Jiwu Huang, Shenzhen MSU-BIT University, Shenzhen, Guangdong, China; e-mail: jwhuang@smbu.edu.cn.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2471-2574/2026/8-ART

<https://doi.org/10.1145/3838797>

1 Introduction

Intelligent voice assistants are among the most commonly deployed sensors within the AIoT ecosystem, finding increasing use in various applications such as autonomous driving [61], smart homes [19], and smart cities [21]. Keyword spotting (KWS), a crucial component of these assistants, identifies specific keywords in continuous audio streams [1]. Widely adopted in popular voice assistants like Siri and Alexa, KWS technology significantly enhances human-computer interaction.

Since local devices typically lack the resources for complex AI tasks, Cloud Voice Assistant (CVA) systems have become highly favored in AIoT scenarios, providing extensive computational and storage services based on audio data [51]. Typically, CVA systems consist of two primary components: (1) a front-end that includes local devices, such as microphones, capturing and uploading audio sensor data, and (2) a backend managed by the cloud services responsible for audio processing tasks, including KWS and speech recognition.

Typically, KWS is implemented as a two-stage process: an on-device lightweight model continuously monitors for predefined wake words (e.g., “Alexa”, “Siri”, or “Google”). Once a wake word is detected, the device captures the subsequent audio stream, which is then transmitted to the cloud. The cloud server performs a more accurate KWS and processes any associated user commands [18, 51].

However, recent studies have shown that KWS models are prone to *misactivations*, including accidental activations triggered by benign sounds and adversarial activations designed by attackers. Prior analyses [18] reported that such misactivations can lead to the leakage of approximately 10 seconds up to one minute of private speech to the service providers. This problem is particularly when local VA devices are installed in private spaces [1], as depicted by the blue area in Fig. 1. Recent incidents, including Alexa-related privacy breaches [24] and unauthorized eavesdropping involving Siri [36], highlight the severity of these concerns. Consequently, developing robust privacy-preserving mechanisms for CVA systems is essential, especially considering the extensive deployment of AIoT microphones.

Significant research has focused on preserving the privacy of sensor data within voice assistant systems. Some studies, such as [1, 15, 23, 43], aim to mitigate risks by device-level controls, limiting microphone access or processing data locally to avoid unauthorized transmissions. Although these methods reduce accidental data leaks, they inherently limit cloud-based service functionality. Specifically, when the cloud servers perform KWS tasks on encrypted speech data, existing device-centric methods necessitate either redesigning lightweight local models or deploying decryption modules, thus increasing complexity and reducing efficiency.

Alternatively, recent cloud-based solutions utilize homomorphic encryption (HE), which provides robust data protection with semantic security, enabling cloud servers to process encrypted data without access to decryption keys. The main research efforts of these approaches focus on designing privacy-preserving deep neural networks (DNNs) under HE constraints, particularly regarding limited multiplicative depth and computational overhead. However, existing privacy-preserving DNN research primarily targets image data, with limited studies addressing KWS tasks within CVA systems. Previous work [60] has focused on designing DNN architectures for KWS within the constraints of multiplicative depth. This study developed a HE-friendly complex-valued convolutional neural network (CNN), which achieved better performance than the compared real-valued CNNs in KWS. However, the exact impact of using complex values to enhance performance remains unclear, as it may lead to parameter redundancy and suboptimal improvements. Conversely, the success of real-valued CNNs for KWS tasks in the plaintext domain [6, 44] demonstrates that real-valued CNNs have the potential to improve performance. Therefore, developing a HE-friendly real-valued CNN architecture that outperforms complex-valued CNNs is a worthwhile endeavor.

This paper introduces APKOS, an Accurate and Privacy-preserving KeywOrd Spotting scheme for CVA systems utilizing HE techniques. We adopt the CKKS (Cheon-Kim-Kim-Song) scheme [13] as the underlying cryptographic primitive to enable computation over encrypted data. Under the dual constraints of semantic security and a limited

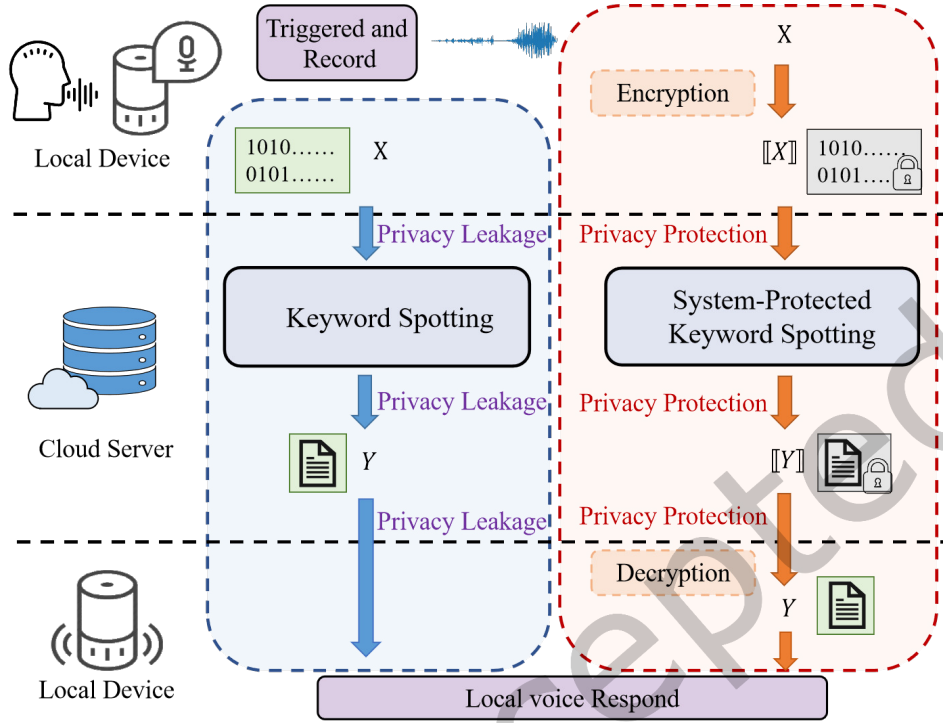


Fig. 1. Conventional KWS solution versus secure KWS solution: the blue branch represents the process of a standard voice application, while the orange branch illustrates that of a secure voice application.

number of homomorphic multiplications, we explore methods to enhance KWS accuracy and robustness. We optimize convolutional kernel sizes and effectively mitigate HE-imposed limitations on network depth, thereby enhancing accuracy under security constraints. We design a real-valued polynomial activation function that maintains superior KWS performance while ensuring training convergence. Moreover, we propose a novel fusion layer combining pooling and activation functionalities, shifting specific ciphertext computations to plaintext operations, thereby improving computational efficiency. Utilizing these secure modules, we develop a HE-friendly residual network architecture for our APKOS scheme. Our approach also supports Single Instruction Multiple Data (SIMD) operations, facilitating batch processing of audio samples.

The primary contributions of this paper are:

- **Better Accuracy:** We propose a novel privacy-preserving KWS scheme for CVA audio data, achieving semantic security and demonstrating superior accuracy compared to existing secure schemes.
- **Improved Efficiency:** We introduce a fusion layer that merges activation and pooling layers, successfully enhancing computational efficiency by converting operations on ciphertexts into operations on plaintexts.
- **Optimized Network Design:** We analyze the impact of kernel size on KWS accuracy under semantic security constraints and design a residual network architecture specifically optimized for privacy-preserving KWS.
- We conduct extensive evaluations demonstrating that APKOS outperforms existing state-of-the-art secure methods by approximately 7% in accuracy and nearly 10% in robustness, setting a new benchmark for privacy-preserving KWS.

The rest of this article is organized as follows. Section 2 reviews related work on privacy-preserving KWS and privacy-preserving deep learning. Section 3 introduces necessary preliminaries, including HE and the security foundations. Section 4 presents the system overview, including the threat model, design goals, and overall workflow. Section 5 details the proposed privacy-preserving KWS scheme. Section 6 provides theoretical analysis and discusses the security guarantees and cryptographic parameters. Section 7 reports experimental results and performance evaluations. Section 8 discusses the limitations and applicability boundaries of the proposed method. In Section 9, we conclude the paper and outline future research directions.

2 Related Work

2.1 Privacy Enhancement in Keyword Spotting Systems

Improving privacy in KWS systems has attracted increasing attention in recent years. In [42, 43], the authors focused on privacy concerns related to the standby state of voice assistants, aiming to prevent unintended recordings. Sikder *et al.* [53] proposed a framework for detecting malicious activities by monitoring changes in sensor data. Meanwhile, other works [1, 15] explored strategies in which microphone data remains local, reducing the risk of both accidental and adversarial activations leaking to external servers. Guimaraes *et al.* [23] used knowledge distillation and geometric constraints to develop robust KWS models under adversarial attacks.

However, these methods primarily address privacy at the device level—either through control strategies (e.g., limiting standby recording) or by preventing data transmission to remote servers. They do not focus on protecting speech data *during processing* in a cloud-based environment. Thus, while they mitigate certain risks, they do not provide a cryptographically secure framework for conducting KWS in outsourced settings. As a result, there remains a clear need for practical solutions that ensure privacy even when speech data is processed remotely, prompting the research direction addressed in this paper.

2.2 Privacy-Preserving Inference of Deep Neural Network

In [22], CryptoNets was designed for implementation over HE ciphertexts by replacing nonlinear activation functions with quadratic functions. Subsequent works [25, 28] explored polynomial approximations of activation functions to improve accuracy. In [37], Li *et al.* introduced a secure CNN prediction scheme based on the fast Fourier Transform. Lou *et al.* [39] designed an HE-friendly CNN architecture for mobile devices, utilizing a greedy architecture search algorithm and coefficient merging. Building upon these early efforts, Lee *et al.* [35] proposed a composite of minimax approximation polynomials to approximate ReLU over HE ciphertexts using bootstrapping. This approach has been applied to several CNN inference schemes over HE ciphertexts [33, 34].

Hybrid approaches [4, 27], which combine FHE with garbled circuits or oblivious transfer, aim to improve performance but are limited by the increased communication costs that scale with the number of layers, hindering their practicality in large-scale applications. In [49], Ran *et al.* used the joint design of model sparsity and data packing to reduce the computational cost of HE operations. Ju *et al.* [30] proposed a new encoding for simultaneous convolutions over HE ciphertexts. Cheon *et al.* [12] introduced a new packing technique which reduces the rotation operations needed for secure CNN implementation. However, the high cost and latency of FHE-based CNN inference remain unresolved, despite these efforts to mitigate the challenges.

While many approaches strive to increase network depth for performance gains, achieving a more efficient solution—even at the cost of lower performance—is often more important in practical applications. Meanwhile, despite extensive research on image-based tasks, there is a notable gap in secure speech processing using CNNs. According to our best knowledge, there are only a few works address this domain. For instance, in [60], Zheng *et al.* presented a complex-valued CNN architecture for secure KWS, but faced issues of inefficiency and inaccuracy. Therefore, researching a more practical and accurate solution for secure KWS is very meaningful.

2.3 Deep Learning-based Keyword Spotting

Early research in KWS predominantly utilized Hidden Markov Models (HMMs) [54, 57], where separate HMMs were trained for each keyword. These systems often suffered from high computational costs and low detection performance.

With the advent of deep learning, the focus shifted to CNN-based KWS. Recent advancements have produced highly accurate networks [46, 48] that incorporate transformer blocks [3, 16], convolutional layers [45], autoregressive layers [50], and various combinations of these architectures [59]. In [47], Ng *et al.* proposed a contrastive speech mixup learning algorithm for low-resource KWS. Vitolo *et al.* [55] proposed a convolutional autoencoder to automatically extract audio features and reduce computational overhead. Cai *et al.* [6] developed a robustness-enhanced KWS system by integrating voice signals and lip-movement features. Navon *et al.* [44] devised an adaptive normalization approach to enhance multi-language KWS, particularly for emerging keywords and low-resource languages.

Despite these remarkable advances in deep learning, the large depth of these network structures poses significant obstacles to their implementation over HE ciphertexts. This challenge motivates the pursuit of specialized architectures of DNN to enable accurate and privacy-preserving KWS over HE ciphertexts.

3 Preliminaries

3.1 Homomorphic Encryption

HE based on RLWE (Ring Learning With Errors) provides security in the post-quantum era [40]. Each homomorphic operation in RLWE-based HE introduces a certain amount of noise into the ciphertext, especially during homomorphic multiplication. To prevent noise overflow, the number of homomorphic multiplications — typically referred to as the multiplicative depth [40] — must be limited. Existing fully homomorphic encryption (FHE) schemes are built upon leveled homomorphic encryption (LHE) and then made fully homomorphic through a bootstrapping procedure, which allows for more homomorphic multiplications. However, current bootstrapping methods are still computationally intensive and impractical for KWS applications. Therefore, adopting LHE as the cryptographic primitive is more practical for secure KWS applications. Examples of LHE include the BGV [5], BFV [20], and CKKS [13] schemes.

In this paper, we employ the CKKS scheme (or its RNS variant [11]) as the underlying LHE. In the following, let $\mathbb{Z}[X]$ denote the ring of polynomials with integer coefficients. We use the notation $\mathcal{R} = \mathbb{Z}[X]/(X^N + 1)$ and $\mathcal{R}_t = \mathcal{R}/2^t\mathcal{R}$, where N is a power-of-two integer. Plaintexts and ciphertexts are defined over $\mathcal{R}$ and $\mathcal{R}_t$, respectively. The CKKS scheme incorporates a complex canonical embedding map, which facilitates the packing of multiple values into a single plaintext polynomial for SIMD batch processing. Consider two vectors $\mathbf{x} = (x_0, x_1, \dots, x_{k-1})$ and $\mathbf{y} = (y_0, y_1, \dots, y_{k-1})$. The SIMD-type homomorphic operations of the CKKS scheme are given as:

$$\text{Dec}(\llbracket \mathbf{x} \rrbracket \oplus \llbracket \mathbf{y} \rrbracket) \approx (x_0 + y_0, x_1 + y_1, \dots, x_{k-1} + y_{k-1}) \quad (1)$$

$$\text{Dec}(\llbracket \mathbf{x} \rrbracket \otimes \llbracket \mathbf{y} \rrbracket) \approx (x_0 \cdot y_0, x_1 \cdot y_1, \dots, x_{k-1} \cdot y_{k-1}) \quad (2)$$

$$\text{Dec}(\llbracket a \odot \mathbf{x} \rrbracket) \approx (ax_0, ax_1, \dots, ax_{k-1}) \quad (3)$$

where $\llbracket \cdot \rrbracket$ and $\text{Dec}(\cdot)$ denote the encryption and the decryption operators, respectively. Each multiplication operation, whether non-scalar (Eq. (2)) or scalar (Eq. (3)), consumes a portion of the available multiplicative depth. Readers are referred to [11, 13] for more detailed constructions of the CKKS scheme.

3.2 Semantic Security

Semantic security, formally known as indistinguishability under chosen-plaintext attacks (IND-CPA), is a fundamental notion of security for encryption schemes. Intuitively, it ensures that the ciphertext reveals no partial

information about the underlying data. In the context of HE, semantic security is critical because encrypted computations must not compromise the confidentiality of the underlying plaintexts. Formally, an HE scheme is said to be semantically secure if, for any two plaintexts m_0 and m_1 chosen by an adversary, the ciphertexts $\llbracket m_0 \rrbracket$ and $\llbracket m_1 \rrbracket$ are computationally indistinguishable. This definition ensures that observing homomorphic computations does not allow adversaries to infer any meaningful information about the plaintext data.

Definition 3.1 (Semantic Security of Homomorphic Encryption). An HE scheme is semantically secure if, for all probabilistic polynomial-time (PPT) adversaries $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that:

$$|\Pr[\mathcal{A}(\llbracket m_0 \rrbracket) = 1] - \Pr[\mathcal{A}(\llbracket m_1 \rrbracket) = 1]| \leq \text{negl}(\lambda) \quad (4)$$

for all pairs of plaintexts m_0, m_1 of the same length, where λ is the security parameter.

The CKKS scheme [13] employed in this paper satisfies semantic security. Specifically, the security of CKKS relies on the hardness of the RLWE problem, which is conjectured to be hard even against quantum adversaries. Therefore, the CKKS scheme ensures that adversaries cannot extract meaningful information from ciphertexts, thereby guaranteeing the confidentiality of speech data during privacy-preserving KWS computations. For further security analysis, readers are referred to [11, 13].

3.3 Simulation-Based Proof

Lindell *et al.* [38] introduced a simulation-based proof method widely utilized to establish the security of secure computation protocols. In this framework, two worlds are considered:

- In the **real world**, the adversary $\mathcal{A}$ executes the actual protocol Π by interacting with the challenger $\mathcal{C}$.
- In the **ideal world**, $\mathcal{A}$ interacts with a probabilistic polynomial-time (PPT) simulator $\mathcal{S}^{\text{PPT}}$ that simulates the behavior of the protocol without access to the real data.

If such a simulator $\mathcal{S}^{\text{PPT}}$ exists that makes the view of $\mathcal{A}$ in the ideal world computationally indistinguishable from its view in the real world, then the protocol Π is deemed secure. Formally, this relationship is expressed as:

$$\text{IDEAL}_{\mathcal{A}, \mathcal{S}^{\text{PPT}}}^{\Pi}(\lambda, \tilde{\xi}) \stackrel{c}{\equiv} \text{REAL}_{\mathcal{A}, \mathcal{C}}^{\Pi}(\lambda, \tilde{\xi}), \quad (5)$$

where λ denotes the security parameter, $\tilde{\xi}$ represents the input of Π , and $\stackrel{c}{\equiv}$ signifies computational indistinguishability. For brevity, we will omit λ and Π in subsequent expressions. We will employ this simulation-based framework to analyze the security of our proposed APKOS under various adversarial models.

4 System Overview

4.1 Threat Model

Fig. 2 illustrates the system model of our secure KWS framework. The user, denoted as $\mathcal{U}$, possesses a large amount of private voice data. Due to limited local resources, $\mathcal{U}$ relies on a cloud service provider, denoted as $\mathcal{CSP}$, for voice data storage and processing.

In our system model, $\mathcal{CSP}$ poses a potential threat to the user's private data. Consistent with common privacy-preserving multimedia processing approaches [10, 29], we consider $\mathcal{CSP}$ as a semi-honest (honest-but-curious) adversary: it follows the prescribed computing protocols and reliably performs the KWS task, yet may attempt to infer sensitive information from the outsourced encrypted speech data and intermediate results (e.g., profiling or analytics). This assumption reflects the cloud voice-assistant setting, where providers are incentivized to maintain stable functionality and avoid reputational or legal and compliance risks, while passive curiosity about user data remains a realistic concern. [58] The “always-on” nature of voice assistants further exacerbates privacy risks, as continuous collection and processing could expose private details.

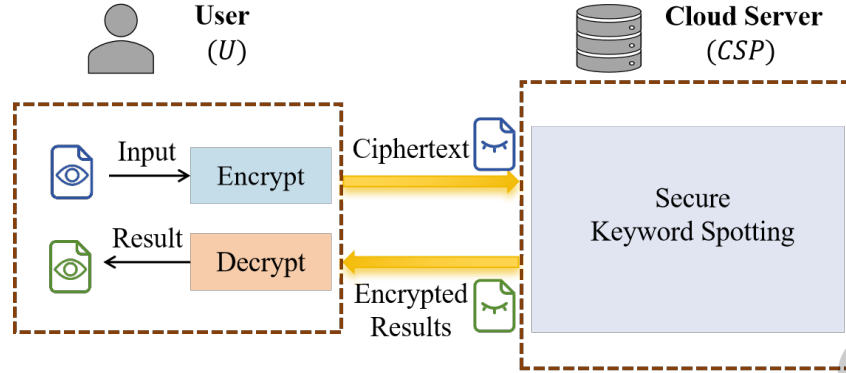


Fig. 2. System model illustrating the interaction between the user (U) and CSP, where CSP processes encrypted speech data without accessing the plaintexts.

Following [10], we evaluate the security under two common threat models, which depend on the level of information accessible to the adversary $\mathcal{A}$:

1) *Known Ciphertext Model*. We assume that $\mathcal{A}$ can access the encrypted speech data and intermediate results during the KWS task execution.

2) *Known Background Model*. This model poses a more serious threat than the known ciphertext model. It assumes that $\mathcal{A}$ has access to additional data, including specific plaintext and ciphertext pairs. This access allows $\mathcal{A}$ to use statistical methods to infer private speech content.

4.2 Design Goals

To address the two threat models, we propose a privacy-preserving KWS scheme built upon RLWE-based HE, enabling secure KWS over encrypted speech. The design of our scheme is guided by the following goals:

- *Speech Privacy Protection*. Following prior works [34, 60], our approach aims to protect users' speech data throughout the entire process of storage and computation. Crucially, it ensures that the confidentiality of speech data is preserved under *semantic security* during cloud-based processing.
- *Efficiency Improvement*. Our scheme aims to reduce runtime on cloud servers compared to the most relevant state-of-the-art privacy-preserving KWS solutions.
- *Performance Improvement*. Compared to state-of-the-art privacy-preserving solutions [33, 34, 60], our scheme outperforms them on key metrics—including accuracy, robustness, F1 score, recall, and precision.

4.3 System Workflow

Based on these design goals, we now present the overall system workflow that integrates local and cloud-side operations to achieve privacy-preserving KWS. Fig. 3 illustrates the overall workflow, which is designed to enhance the privacy protection of VA systems. The workflow aligns with the typical two-stage KWS process employed in modern VAs, such as Alexa, Siri, and Google Assistant.

- (1) **Local Wake Word Detection**. The user's smart VA device continuously samples ambient audio through its microphone, monitoring for the presence of predefined wake words (e.g., "Alexa", "Siri", or "Google"). Although this local KWS model is lightweight for low latency, it comes at the cost of reduced accuracy [52].

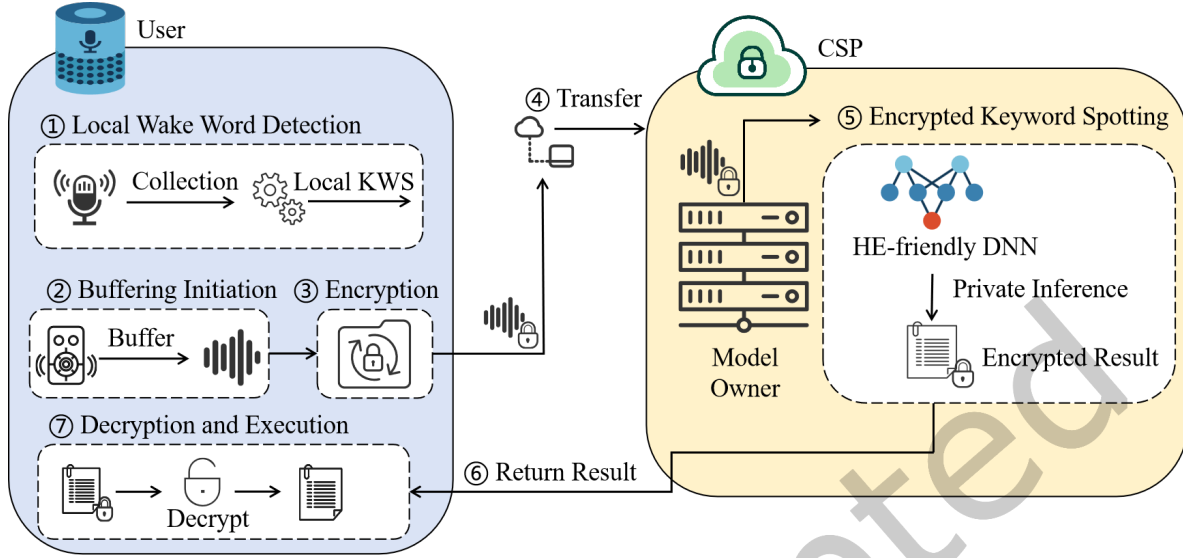


Fig. 3. Overall workflow of privacy-preserving KWS in intelligent voice assistant systems.

- (2) **Wake Word Detected.** Once the local model detects a wake word, it triggers the start of an audio recording session. The device begins buffering subsequent voice streams, typically capturing several seconds of audio [18].
- (3) **Encryption of the Recorded Audio.** To protect user privacy, the buffered audio is immediately encrypted using the CKKS scheme before leaving the device. This encryption ensures that sensitive voice content is secured under semantic security guarantees.
- (4) **Transmission of Encrypted Audio to the Cloud.** The encrypted audio data is transmitted to the cloud (CSP) for further processing. Since the data remains encrypted throughout transmission, CSP cannot access or infer any information from the raw speech content.
- (5) **Encrypted Keyword Spotting in the Cloud.** Upon receipt, CSP performs a second-stage KWS verification on the encrypted audio using our proposed HE-friendly DNN. This cloud-based model, which is more sophisticated and accurate than the device-side model, determines whether a true wake word was spoken and processes any associated user commands—all without decrypting the audio data.
- (6) **Returning the Inference Results.** After completing KWS verification and any necessary command inference, CSP sends the encrypted results back to the user device.
- (7) **Decryption and Final Action Execution.** The user device decrypts the received results using the private decryption key and proceeds to execute the corresponding user request or interaction, if applicable.

This workflow addresses a critical privacy vulnerability in existing VA systems. Studies [18, 51] have shown that misactivations—including accidental activations caused by benign sounds and adversarial activations crafted by attackers—can lead to the unintended recording and uploading of private conversations to the cloud. Prior evaluations [18] report that a single misactivation could result in approximately 10 seconds of private speech leakage. In [1], the authors indicate that certain misactivations may expose up to one minute of private speech to the cloud server.

By introducing HE at the point of wake word detection, our system ensures that all audio data sent to CSP remains confidential under semantic security, effectively mitigating privacy risks arising from both accidental and

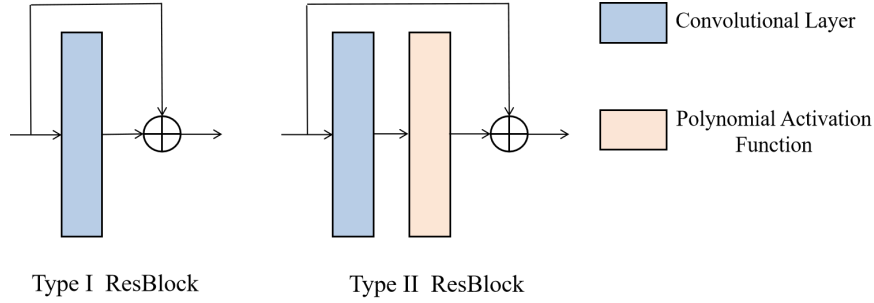


Fig. 4. Two types of residual blocks over HE ciphertexts. Type I ResBlock applies skip connection immediately after the convolution, whereas Type II ResBlock introduces a polynomial activation function between the convolution and the residual addition.

adversarial misactivations [1, 7]. Thus, even if misactivations occur at the device level, CSP remains incapable of accessing or exploiting the private speech content.

5 Accurate and Privacy-Preserving Keyword Spotting Scheme with Semantic Security

We present the core encrypted inference components of APKOS, our accurate and privacy-preserving KWS system tailored for CVA applications. At the core of APKOS lies CryptKWSNet, the HE-friendly CNN architecture tailored for privacy-preserving KWS tasks. CryptKWSNet is specifically designed to support privacy-preserving inference while preserving the semantic security of user speech data. This ensures that the cloud-based processing of encrypted audio does not reveal any meaningful information to CSP. Our CryptKWSNet enables batched processing of encrypted inputs, leveraging the SIMD capabilities of RLWE-based HE schemes to enhance computational efficiency. The following subsections detail the main building blocks of CryptKWSNet: a convolutional layer, a residual layer, a real-valued polynomial approximation activation function, and a fusion layer. We then describe our kernel expansion strategy and its role in improving representational capacity and overall KWS performance.

5.1 Convolutional Layer and Residual Layer over HE Ciphertexts

We first consider multi-output convolutions with multiple input channels. Assume there are N_{out} filters, where each filter consists of N_{in} kernels (one for each input channel). For each filter, a single-channel convolution with an input channel produces an encrypted vector. Given N_{in} input channels and N_{out} filters, this process yields N_{in} encrypted vectors. Summing these vectors generates an output encrypted vector. Repeating this for all filters, we obtain an output matrix with N_{out} encrypted vectors. The input matrix $\mathcal{X}$ consists of N_{in} encrypted vectors, i.e., $\mathcal{X} = (\mathbf{v}_1^\top, \mathbf{v}_2^\top, \dots, \mathbf{v}_{N_{in}}^\top)^\top$, where $()^\top$ indicates matrix transposition and $\mathbf{v}_j = (\llbracket \mathbf{x}_{j,1} \rrbracket, \llbracket \mathbf{x}_{j,2} \rrbracket, \dots, \llbracket \mathbf{x}_{j,n} \rrbracket)$. We use $\mathcal{X}_{j,l}$ to denote the (j, l) -th element in $\mathcal{X}$, i.e., $\llbracket \mathbf{x}_{j,l} \rrbracket$. The output is also a matrix $\tilde{\mathcal{X}}$ with N_{out} vectors of ciphertexts.

Since the HE algorithm only supports homomorphic addition and multiplication, non-arithmetic activation functions cannot be exactly executed over HE ciphertexts. Existing HE-based security work uses polynomial functions as activation functions. However, CNNs employing polynomial activation functions can experience instability during training [8]. This instability, caused by the introduction of very large gradient values in certain regions, prevents network training convergence. To address this challenge, we incorporate a residual structure. This structure allows the input of upper feature maps to be partially included in the output through shortcut connections, ensuring lossless data transmission. When the convolutional part underperforms, the original data is seamlessly integrated into the output via the shortcut. Conversely, when the convolutional part is

Algorithm 1 Privacy-preserving convolution and residual layers over HE ciphertexts.

Input: Convolution kernels $C_{jqp} \in \mathbb{R}^k$; input matrix $\mathcal{X}$ of N_{in} ciphertext vectors; polynomial coefficients $a_0, a_1, \dots, a_n \in \mathbb{R}$; layer type flag $\tau \in \{0, 1, 2\}$ $\triangleright \tau = 0$: ConvLayer, $\tau = 1$: Type I ResBlock, $\tau = 2$: Type II ResBlock

Output: Output matrix $\tilde{\mathcal{X}}$ of N_{out} ciphertext vectors

```

1: Precompute:  $ct_0 \leftarrow \llbracket 0 \rrbracket, ct_1 \leftarrow \llbracket 1 \rrbracket$   $\triangleright$  Encrypted zero and one
2: for  $j = 1, 2, \dots, N_{out}$  do
3:   for  $i = 1, 2, \dots, n - k + 1$  do
4:      $\tilde{\mathcal{X}}_{ji} \leftarrow ct_0$ 
5:     for  $q = 1, 2, \dots, N_{in}$  do
6:       for  $p = 1, 2, \dots, k$  do
7:          $\tilde{\mathcal{X}}_{ji} \leftarrow \tilde{\mathcal{X}}_{ji} \oplus (C_{jqp} \odot \mathcal{X}_{q,i+p-1})$ 
8:       end for
9:     end for
10:    if  $\tau = 0$  or  $\tau = 2$  then  $\triangleright$  Apply polynomial activation if needed
11:       $ct_{acc} \leftarrow a_0 \odot ct_1$   $\triangleright$  Initialize accumulator
12:       $ct \leftarrow \tilde{\mathcal{X}}_{ji}$   $\triangleright$  Current power of  $x$ 
13:      for  $d = 1, \dots, n$  do
14:         $ct_{acc} \leftarrow ct_{acc} \oplus (a_d \odot ct)$ 
15:         $ct \leftarrow ct \otimes \tilde{\mathcal{X}}_{ji}$   $\triangleright$  Compute next power
16:      end for
17:       $\tilde{\mathcal{X}}_{ji} \leftarrow ct_{acc}$ 
18:    end if
19:  end for
20: end for
21: if  $\tau = 0$  then  $\triangleright$  Convolution-only layer
22:   return  $\tilde{\mathcal{X}}$ 
23: else if  $\tau = 1$  then  $\triangleright$  Type I: Conv + residual
24:    $\tilde{\mathcal{X}} \leftarrow \tilde{\mathcal{X}} \oplus \mathcal{X}$ 
25:   return  $\tilde{\mathcal{X}}$ 
26: else if  $\tau = 2$  then  $\triangleright$  Type II: Conv + activation + residual
27:    $\tilde{\mathcal{X}} \leftarrow \tilde{\mathcal{X}} \oplus \mathcal{X}$ 
28:   return  $\tilde{\mathcal{X}}$ 
29: end if

```

effective, the output still retains the original data, allowing for further refinement by subsequent convolutional layers. We propose two types of residual structures, illustrated in Fig. 4. In the Type I ResBlock, the output $\text{ResBlock}_1(\mathcal{X})$ is the homomorphic sum of the current input $\mathcal{X}$ and the convolutional layer output $\text{Convolution}(\mathcal{X})$, i.e., $\text{ResBlock}_1(\mathcal{X}) = \text{Convolution}(\mathcal{X}) + \mathcal{X}$. In contrast, the Type II ResBlock further applies a polynomial activation function $\text{PActivation}(\cdot)$ to the convolutional result before the residual connection, yielding $\text{ResBlock}_2(\mathcal{X}) = \text{PActivation}(\text{Convolution}(\mathcal{X})) + \mathcal{X}$. The details of the convolutional and residual layers are presented in Algorithm 1.

5.2 Activation Function over HE Ciphertexts

Activation functions are crucial for the effectiveness of CNNs over HE ciphertexts. The selection and accurate implementation of appropriate activation functions can significantly improve KWS performance. The commonly used ReLU function poses a challenge when implemented over HE ciphertexts due to its non-arithmetic nature. It is difficult to implement it accurately using the CKKS scheme. Previous studies [33, 34] approximate ReLU using composite polynomials by minimax approximation. However, this method is unfeasible without bootstrapping, making it unsuitable for the efficient, non-bootstrapping HE schemes considered in this paper.

Several studies have investigated polynomial approximations in bootstrapping-independent schemes. In [22], Gilad-Bachrach *et al.* suggested using a quadratic function x^2 as the activation function. However, using a quadratic function is suboptimal in many CNN models due to the large difference between x^2 and conventional activation functions. In [8, 25], the popular activation function ReLU is approximated as polynomial ReLU (PReLU) using methods such as least squares or derivative approximation.

Recently, PReLU_C, a complex-valued polynomial approximation designed for complex-valued CNNs, was introduced in [60]. Specifically, it is defined as $\text{PReLU}_C = 6.835 \times 10^{-6}z^3 + 0.234z^2 + 0.499z + 0.187$, where z is a complex-valued variable. However, in real-valued CNNs for KWS, model weights are real numbers rather than complex numbers. Therefore, z must be replaced with a real-valued variable x when applying PReLU_C for real-valued CNNs. Our investigation reveals that PReLU_C is suboptimal for real-valued CNNs in KWS, particularly in terms of training stability and model performance. To address this limitation, we propose a new real-valued polynomial approximation of ReLU. Given the non-differentiability of ReLU at zero and the influence of polynomial order on approximation accuracy, careful selection of approximation intervals is essential. Unlike PReLU_C, which approximates ReLU within the interval $[-1, 1]$, we perform the approximation over a wider interval, $[\alpha, \beta]$, to better align with the input range. For example, when we instantiate $[\alpha, \beta]$ as $[-3, 3]$, the resulting polynomial PReLU_R becomes $1.275 \times 10^{-6}x^3 + 0.134x^2 + 0.499x + 0.281$. Although this introduces slightly larger deviations near zero, it achieves a more accurate overall fit to ReLU across the broader interval. As a result, PReLU_R offers improved numerical stability, particularly in the negative input region. With these characteristics, PReLU_R enhances the robustness of real-valued CNNs used in KWS tasks, especially when trained on large-scale datasets. Further analysis and experimental results are provided in Section 7.3.1.

5.3 Pooling Layer and Fusion Layer over HE Ciphertexts

The CKKS scheme does not support comparison operations, so average pooling is usually used to downsample the input ciphertexts [60]. Consider a one-dimensional average pooling layer as an example. Given an input ciphertext $\mathcal{X} = (v_1^\top, v_2^\top, \dots, v_{N_{in}}^\top)^\top$, the pooling layer slides a window of size k over the ciphertext, with a stride of k . At each window position, it computes the average of q ciphertexts, where $q \leq k$. The output comprises $\lfloor \frac{N_{in}}{k} \rfloor$ encrypted vectors, where $\lfloor \cdot \rfloor$ denotes the floor operation. The specific procedures of the pooling layer are detailed in Algorithm 2.

From Algorithm 2, we know that for an input of N_{in} ciphertexts, a single pooling operation with kernel size k requires $\lfloor \frac{N_{in}}{k} \rfloor k$ scalar multiplications. This computational burden is directly proportional to the size of k . An improved approach is to sum the values in each pooling window before averaging, i.e., $\llbracket \tilde{x}'_i \rrbracket = \frac{1}{k} \odot (\llbracket x'_i \rrbracket \oplus \llbracket x'_{i+1} \rrbracket \oplus \dots \oplus \llbracket x'_{i+k-1} \rrbracket)$. However, this improved method still requires a total of $\lfloor \frac{N_{in}}{k} \rfloor$ scalar multiplications.

Optimizing the computational complexity of a pooling layer alone is very challenging. Therefore, we turn to the joint optimization of the two network layers with the goal of reducing the overall computational load of the two network layers. Sequential combinations of activation functions and average pooling layers are common in privacy-preserving CNN schemes [33, 34, 60]. We focus on analyzing the computational load required by these two network layers. The process of calculating $\llbracket x \rrbracket$ by the pooling layer and the activation function layer is as follows:

Algorithm 2 Privacy-preserving pooling layer over HE ciphertexts.**Input:** Encrypted inputs $\llbracket \mathbf{x}_1 \rrbracket, \dots, \llbracket \mathbf{x}_{N_{in}} \rrbracket$; pooling window size $k \in \mathbb{Z}^+$ **Output:** Encrypted pooled outputs $\llbracket \tilde{\mathbf{x}}_1 \rrbracket, \dots, \llbracket \tilde{\mathbf{x}}_{\lfloor N_{in}/k \rfloor} \rrbracket$

```

1:  $\text{ct}_0 \leftarrow \llbracket 0 \rrbracket$  ▷ Precompute encrypted constant
2: for  $j = 1$  to  $\lfloor N_{in}/k \rfloor$  do
3:    $\llbracket \tilde{\mathbf{x}}_j \rrbracket \leftarrow \text{ct}_0$  ▷ Initialize accumulator
4:   for  $l = (j-1)k$  to  $jk-1$  do
5:      $\llbracket \tilde{\mathbf{x}}_j \rrbracket \leftarrow \llbracket \tilde{\mathbf{x}}_j \rrbracket \oplus (\frac{1}{k} \odot \llbracket \mathbf{x}_l \rrbracket)$  ▷ Homomorphic average
6:   end for
7: end for
8: return  $\llbracket \tilde{\mathbf{x}}_1 \rrbracket, \dots, \llbracket \tilde{\mathbf{x}}_{\lfloor N_{in}/k \rfloor} \rrbracket$ 

```

$$\text{Activation Function Layer : } \llbracket \mathbf{x}'_i \rrbracket = a_0 \oplus a_1 \odot \llbracket \mathbf{x}_i \rrbracket \oplus a_2 \odot \llbracket \mathbf{x}_i \rrbracket^2 \oplus a_3 \odot \llbracket \mathbf{x}_i \rrbracket^3, \quad (6)$$

$$\text{Pooling Layer : } \llbracket \tilde{\mathbf{x}}'_i \rrbracket = \frac{1}{k} \odot (\llbracket \mathbf{x}'_i \rrbracket \oplus \llbracket \mathbf{x}'_{i+1} \rrbracket \oplus \dots \oplus \llbracket \mathbf{x}'_{i+k-1} \rrbracket). \quad (7)$$

The activation function layer has a multiplicative depth of two, involving $2N_{in}$ non-scalar multiplications (Eq. (2)) and N_{in} scalar multiplications (Eq. (3)). The pooling layer requires one multiplicative depth, necessitating at least $\lfloor \frac{N_{in}}{k} \rfloor$ scalar multiplications. These two network layers together require three multiplicative depths, performing at least $2N_{in}$ non-scalar multiplications and $(N_{in} + \lfloor \frac{N_{in}}{k} \rfloor)$ scalar multiplications.

To jointly optimize the computational load of the two network layers, we propose a new network layer, the fusion layer. It combines the functions of the activation and pooling layers into a single unified network layer. The proposed fusion layer merges the pooling layer with the activation function layer in the encrypted domain. Specifically, by merging the constant coefficient of the polynomial activation function with the average of the pooling layer, one multiplication operation can be saved. Note that the order of the pooling and activation functions can be interchanged without affecting the effectiveness of the fusion layer. Mathematically, the proposed fusion layer can be represented as follows:

$$\text{Fusion Layer : } \llbracket \mathbf{x}'_i \rrbracket = \frac{a_0}{k} \oplus \frac{a_1}{k} \odot \llbracket \mathbf{x}_i \rrbracket \oplus \frac{a_2}{k} \odot \llbracket \mathbf{x}_i \rrbracket^2 \oplus \frac{a_3}{k} \odot \llbracket \mathbf{x}_i \rrbracket^3, \quad (8)$$

$$\llbracket \tilde{\mathbf{x}}'_i \rrbracket = \llbracket \mathbf{x}'_i \rrbracket \oplus \llbracket \mathbf{x}'_{i+1} \rrbracket \oplus \dots \oplus \llbracket \mathbf{x}'_{i+k-1} \rrbracket. \quad (9)$$

From the mathematical expression above, we observe that the $\frac{1}{k}$ scaling operation is moved from the pooling layer to the activation function layer. Thus, we have successfully converted the expensive homomorphic multiplication into fast conventional multiplication, saving computational overhead. Since the pooling operations no longer consume multiplicative depth, the fusion layer has a multiplicative depth of two, performing $2N_{in}$ non-scalar multiplications and N_{in} scalar multiplications. The detailed steps of the fusion layer are provided in Algorithm 3. Note that we use ct_0 and ct_1 to denote the encrypted representations of 0 and 1, respectively; these ciphertexts are reused throughout the computation to avoid redundant encryption. As shown in line 7 of Algorithm 3, the exponentiation of ciphertexts can be optimized using the square-and-multiply method to reduce the number of homomorphic multiplications.

Algorithm 3 Privacy-preserving fusion layer with n -degree polynomial activation**Input:** Encrypted inputs $\llbracket x_1 \rrbracket, \dots, \llbracket x_{N_{in}} \rrbracket$; polynomial coefficients $a_0, \dots, a_n$; pooling window size $k \in \mathbb{Z}^+$ **Output:** Encrypted outputs after fused activation and pooling $\llbracket \tilde{x}_1 \rrbracket, \dots, \llbracket \tilde{x}_{\lfloor N_{in}/k \rfloor} \rrbracket$

```

1:  $ct_0 \leftarrow \llbracket 0 \rrbracket, ct_1 \leftarrow \llbracket 1 \rrbracket$  ▷ Precompute encrypted constants
2: for  $i = 1$  to  $N_{in}$  do
3:    $\llbracket x_i^{(0)} \rrbracket \leftarrow ct_1$  ▷ Initialize  $x^{(0)}$ 
4:    $\llbracket x'_i \rrbracket \leftarrow ct_0$  ▷ Initialize polynomial accumulator
5:   for  $d = 0$  to  $n$  do
6:     if  $d > 0$  then
7:        $\llbracket x_i^{(d)} \rrbracket \leftarrow \llbracket x_i^{(d-1)} \rrbracket \otimes \llbracket x_i \rrbracket$  ▷ Can be optimized via square-and-multiply
8:     end if
9:      $\llbracket x'_i \rrbracket \leftarrow \llbracket x'_i \rrbracket \oplus \left( \frac{a_d}{k} \odot \llbracket x_i^{(d)} \rrbracket \right)$  ▷ Scaled coefficient accumulation
10:   end for
11: end for
12: for  $j = 1$  to  $\lfloor N_{in}/k \rfloor$  do
13:    $\llbracket \tilde{x}_j \rrbracket \leftarrow \llbracket x'_{(j-1)k+1} \rrbracket$  ▷ Initialize pooling window sum
14:   for  $l = (j-1)k + 2$  to  $jk$  do
15:      $\llbracket \tilde{x}_j \rrbracket \leftarrow \llbracket \tilde{x}_j \rrbracket \oplus \llbracket x'_l \rrbracket$ 
16:   end for
17: end for
18: return  $\llbracket \tilde{x}_1 \rrbracket, \dots, \llbracket \tilde{x}_{\lfloor N_{in}/k \rfloor} \rrbracket$ 

```

5.4 Kernel Expansion and Network Architecture: A Speech-Specific Design

Privacy-preserving CNNs typically consist of stacked convolutional blocks, including convolutional, activation, and pooling layers [60]. Increasing the network depth is the typical approach to improve the performance of KWS in the plaintext domain. However, existing plaintext-domain KWS models are often too deep to be feasibly implemented in the encrypted domain without bootstrapping. Therefore, improving the performance of KWS without increasing network depth—while avoiding bootstrapping to maintain computational efficiency—remains challenging.

Why kernel expansion is speech-specific. Unlike images, speech time–frequency features (e.g., log-Mel spectrograms) are strongly *anisotropic* and time-ordered: the two axes have distinct physical meanings. Discriminative cues for wake words are primarily distributed along the temporal axis and often span multiple frames, while translation invariance along the frequency axis is more limited (e.g., shifting formant-related patterns can change phonetic identity). In the plaintext domain, such long-range temporal dependencies are usually modeled by deeper networks (more stacked layers). However, in the HE domain, depth is tightly limited by multiplicative depth and bootstrapping cost. Kernel expansion offers a speech-oriented alternative: enlarging temporal convolution kernels (equivalently, using $1 \times K$ kernels on a time–frequency map, or 1D temporal convolutions by treating frequency bins as channels) increases the temporal receptive field directly, enabling the model to capture longer phonetic patterns and improving robustness under noisy conditions, without increasing network depth. Moreover, restricting expansion to the temporal dimension keeps the growth of ciphertext operations more manageable than isotropic 2D expansion, which would inflate computation and memory footprints under HE constraints [17, 60].

To address this issue, we investigate the impact of kernel size on KWS performance under a fixed network depth. The size of a convolutional kernel determines the receptive field of neurons in a given layer. For the KWS task, the kernel size affects the temporal context perceived at each time step. Increasing the range of features

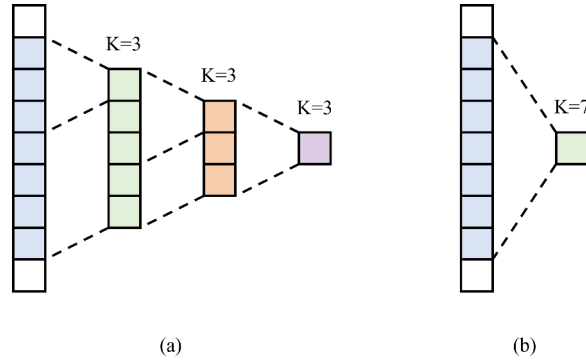


Fig. 5. The relationship between convolution kernel size and the range of features captured. On the left are the feature ranges captured by three convolution kernels each with a size of 3. On the right is the feature range captured by a single convolution kernel with a size of 7.

captured by each kernel allows the model to better exploit contextual information and long-range dependencies. Fig. 5 illustrates the relationship between kernel size and the effective receptive field. As shown, the feature range captured by three consecutive kernels of size 3 is comparable to that of a single kernel of size 7.

We propose using kernel expansion to compensate for the limitations of multiplicative depth. Starting from a basic privacy-preserving CNN scheme with small-kernel convolutional layers, we perform a first kernel expansion by increasing the kernel size of each convolutional layer. This enhances the model’s capacity to capture complex features without increasing the network depth. Then, we apply a second kernel expansion to highlight the contribution of shallow-layer features by enlarging the kernel size of the first convolutional layer, allowing early layers to aggregate longer-span temporal cues.

With this two-stage kernel expansion strategy, we design a new real-valued CNN architecture to improve performance on the KWS task. In summary, unlike prior approaches that focus on increasing network depth [34, 60], our method emphasizes using larger kernels to capture features equivalent to those extracted by deeper networks with smaller kernels. This strategy improves the robustness of privacy-preserving KWS systems operating over HE ciphertexts.

6 Theoretical Analysis and Discussions

6.1 Security Analysis

Our security analysis adopts the semi-honest (honest-but-curious) cloud adversary, consistent with the threat model in Section 4.1. In commercial CVA deployments, the provider is typically incentivized to execute inference correctly to maintain service quality, whereas the more realistic risk is *passive inference* from encrypted traces and intermediate results. In contrast, defending against a fully malicious cloud (arbitrary deviations such as malformed computation or selective aborts) generally requires additional verification/authentication mechanisms and incurs substantial overhead [26], which is orthogonal to our goal of practical always-on KWS without bootstrapping. We therefore focus on rigorous confidentiality guarantees against passive leakage during outsourced HE computation.

We use the simulation-based proof method to analyze the security of our APKOS scheme. In a simulation-based proof, a system is secure if an adversary, $\mathcal{A}$, cannot distinguish between the view of a challenger, $\mathcal{C}$, in the real world and a simulator, $\mathcal{S}$, in the ideal world. For clarity, let us denote the encrypted version of the challenge user’s plaintext audio as $\mathbb{E}_{\text{ct}}$. The security of the framework during initialization and data encryption can be directly inferred based on the semantic security guarantees provided by the CKKS scheme. Since CKKS provides

semantic security, both phases are secure. Therefore, we focus on analyzing the security of HE computations performed on the cloud server, following the two threat models outlined in Section 4.1.

THEOREM 6.1. *APKOS is secure against the cloud server under the known ciphertext model.*

PROOF. During the cloud computing phase, the server performs a series of homomorphic operations on $\mathbb{E}_{ct}$ to compute the result. Assume $\mathcal{A}_{CSP}^{SH}$ is a semi-honest adversary operating the cloud in the real world. The view of $\mathcal{A}_{CSP}^{SH}$ encompasses the encrypted speech data $\mathbb{E}_{ct}$, all encrypted using the CKKS scheme.

A simulator $\mathcal{S}^{PPT}$ is constructed in the ideal world to replicate the view of $\mathcal{A}_{CSP}^{SH}$ in the real world. $\mathcal{S}^{PPT}$ can simulate the ciphertexts using an equivalent number of random polynomial pairs, and returns these to $\mathcal{A}_{CSP}^{SH}$. If $\mathcal{A}_{CSP}^{SH}$ could differentiate between the real world view and that of the ideal world, it implies an ability to distinguish between a CKKS ciphertext and a random polynomial pair, contradicting the semantic security of the CKKS scheme. Consequently, the cloud server cannot distinguish between the ideal and real worlds, as expressed by:

$$\text{IDEAL}_{\mathcal{A}_{CSP}^{SH}, \mathcal{S}^{PPT}}(\tilde{\xi}) \stackrel{c}{\equiv} \text{REAL}_{\mathcal{A}_{CSP}^{SH}, C}(\tilde{\xi}), \quad (10)$$

where $\tilde{\xi} = \{\mathbb{E}_{ct}\}$. Therefore, under the known ciphertext model, the adversary is unable to extract any private information from the encrypted data and the computed results. $\square$

THEOREM 6.2. *APKOS is secure against the cloud server under the known background model.*

PROOF. Under the known background model, a semi-honest cloud server can accumulate a subset of users' plaintext audio data and corresponding ciphertexts, forming a collection of plaintext-ciphertext pairs $\bigcup_i \{[a_i], a_i\} \triangleq \mathbb{A}$. Assume $\mathcal{A}_{CSP}^{SH}$ is a semi-honest adversary operating the cloud in the real world. The view of $\mathcal{A}_{CSP}^{SH}$ includes the encrypted speech data $\mathbb{E}_{ct}$ and the assembled collection of plaintext-ciphertext pairs $\mathbb{A}$. $\mathcal{A}_{CSP}^{SH}$ may perform homomorphic operations on $\mathbb{A}$ and $\mathbb{E}_{ct}$ in an attempt to decipher private information.

A simulator $\mathcal{S}^{PPT}$ is created in the ideal world to emulate the view of $\mathcal{A}_{CSP}^{SH}$ in the real world. $\mathcal{S}^{PPT}$ simulates $\mathbb{E}_{ct}$ by generating an equal number of random polynomial pairs, and simulates $\mathbb{A}$ by preserving the original plaintexts while replacing the corresponding ciphertexts with random polynomial pairs. The simulator then presents these to $\mathcal{A}_{CSP}^{SH}$. $\mathcal{A}_{CSP}^{SH}$ cannot distinguish the simulated ciphertexts from genuine CKKS ciphertexts; otherwise, this would contradict the semantic security of the CKKS scheme. Thus, the cloud server is unable to distinguish between the ideal world and the real world, as stated:

$$\text{IDEAL}_{\mathcal{A}_{CSP}^{SH}, \mathcal{S}^{PPT}}(\tilde{\xi}) \stackrel{c}{\equiv} \text{REAL}_{\mathcal{A}_{CSP}^{SH}, C}(\tilde{\xi}), \quad (11)$$

where $\tilde{\xi} = \{\mathbb{A}, \mathbb{E}_{ct}\}$. Hence, the adversary cannot acquire any private information under the known background model. $\square$

6.2 Analysis of the Precision of the Processed Results under Cryptographic Parameters

We use N to denote the degree of the polynomial modulus, Q for the initialized ciphertext modulus, and Δ as the scaling factor. When the multiplicative depth increases, Q needs to be correspondingly larger. To maintain a comparable level of security in the CKKS scheme, increasing N is necessary, but this also results in higher computational costs. Balancing the computational overhead at the minimum multiplicative depth is crucial.

In particular, in the CKKS scheme, the plaintext message is multiplied by Δ . We denote the pair of its ciphertext and this scaling factor value as (ct, Δ) . When two ciphertexts ct_1 and ct_2 are multiplied, the result of the pair is $(ct_1 \otimes ct_2, \Delta^2)$. To maintain a constant scaling ratio and reduce noise in the ciphertext, the rescale operation is performed after homomorphic multiplications.

We compute the multiplicative depth required by the proposed APKOS. In particular, the convolutional, average pooling, and fully connected layers perform only scalar multiplications, whereas the fusion and residual layers

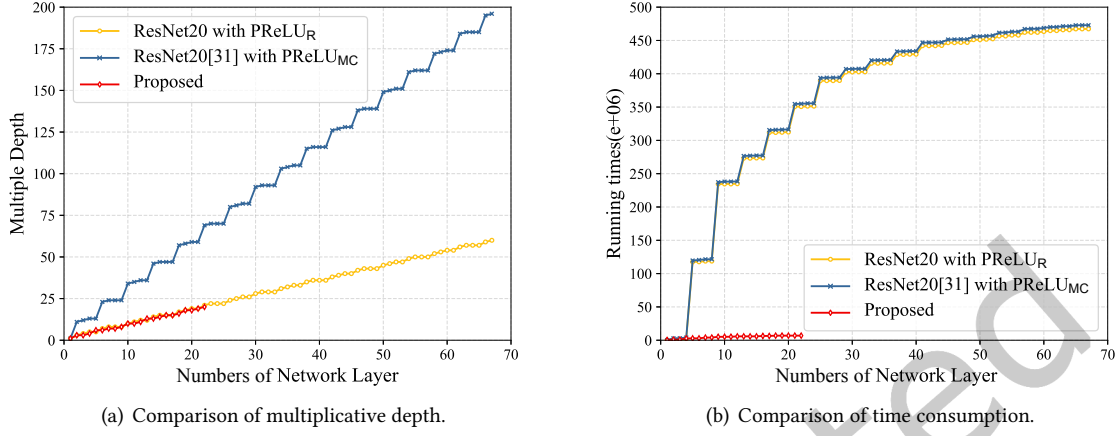


Fig. 6. Comparison of multiplicative depth and time consumption between the conventional ResNet-20 (with bootstrapping) and the proposed model (without bootstrapping).

perform both scalar and non-scalar multiplications. We use p_c and p to denote the number of scalar and non-scalar multiplications, respectively, and denote $\vartheta = (p_c, p)$. The convolutional, average-pooling, and fully-connected layers each require $\vartheta_{\text{conv}} = (1, 0)$, $\vartheta_{\text{avgpool}} = (1, 0)$, and $\vartheta_{\text{fc}} = (1, 0)$, respectively; the fusion layer requires $\vartheta_{\text{fusion}} = (2, 1)$; and the Type I and Type II residual layers require $\vartheta_{\text{res-1}} = (1, 0)$ and $\vartheta_{\text{res-2}} = (1, 1)$, respectively. The total number of homomorphic multiplications in our APKOS is:

$$\vartheta_{\text{all}} = 3(\vartheta_{\text{conv}} + \vartheta_{\text{fusion}}) + 2(\vartheta_{\text{res-1}} + \vartheta_{\text{res-2}}) + \vartheta_{\text{avgpool}} + \vartheta_{\text{fc}} = (15, 5).$$

We use Q_0 to denote the modulus required for the precision of the integer and fractional parts. The minimum Q can be computed as $Q = \Delta^L Q_0$, where $L = \|\vartheta_{\text{all}}\|_1$. For instance, if we desire 30-bit precision for the fractional part and 10-bit for the integer part, then $\Delta = 2^{30}$ and $Q_0 = 2^{10} \cdot 2^{30} = 2^{40}$. To achieve a 128-bit security level for our APKOS, we can set $N = 2^{11}$ and $Q = 2^{800}$. Then, we have

$$Q_0 = \frac{Q}{\Delta^L} = \frac{Q}{\Delta^{\|\vartheta_{\text{all}}\|_1}} = \frac{2^{800}}{2^{30(15+5)}} = 2^{200}, \quad (12)$$

which means that this setting leaves 200 bits of modulus for the ciphertext. This is enough to guarantee that the final results of our APKOS maintain at least 40 bits of precision after decryption.

An Example of Network Depth Exceeding the Multiplicative Depth Limit. We provide a simple example to illustrate the impact of ciphertext multiplication limitations on CNN structure design. Consider CryptoNet [22] as an example. CryptoNet consists of five layers: the first and third layers each contain a convolution layer followed by an activation function layer, and the fifth layer is a fully connected layer. Each convolution and fully connected layer involves one scalar multiplication, while each square activation function involves one non-scalar multiplication. Accumulating these across the five layers yields a total multiplicative depth of 5. If the underlying HE scheme supports only a multiplicative depth of 4, decryption of the processed results from CryptoNet will produce erroneous outputs. Although in many practical settings the multiplicative depth supported by HE schemes may exceed 4, it is still fixed. This fixed depth inherently constrains the design of KWS networks, making it infeasible to arbitrarily increase the network depth.

6.3 Computational Complexity Analysis

To rigorously quantify the efficiency gains of APKOS, we analyze the computational cost of each key component in terms of homomorphic operations. We denote the number of input and output channels as N_{in} and N_{out} , respectively, and the kernel size as k . Table 1 details the breakdown of scalar multiplications (Plaintext-Ciphertext), non-scalar multiplications (Ciphertext-Ciphertext), and homomorphic additions. The associated Multiplicative Depth Cost is denoted as a tuple $\vartheta = (p_c, p)$, where p_c and p to denote the number of scalar and non-scalar multiplications.

Table 1. Complexity Breakdown of Key APKOS Components.

Component	No. Scalar Mult.	No. Cipher Mult.	No. Additions	Depth Cost
Conv Layer	$N_{out} \cdot N_{in}$	0	$N_{out} \cdot N_{in} \cdot k$	(1,0)
FC Layer	$N_{out} \cdot N_{in}$	0	$N_{out} \cdot N_{in}$	(1,0)
Activation+pooling	$N_{out} \cdot k$	$3N_{in}$	$3N_{in} + N_{out} \cdot k$	(3,1)
Fusion Layer	N_{out}	$2N_{in}$	$3N_{in} + N_{out} \cdot k$	(2,1)
Type I ResBlock	$N_{out} \cdot N_{in}$	0	$N_{out}(N_{in}(k+1)+1)$	(1,0)
Type II ResBlock	$N_{out} \cdot N_{in}$	$2N_{out}$	$N_{out}(N_{in}(k+1)+3)$	(1,1)

Note: 1) Calculation time for unit operations: Scalar Mult. ≈ 2.33 ms, Cipher Mult. ≈ 9.87 ms, Add. ≈ 0.73 ms. 2) Weights in Conv, FC, and Type I ResBlock are plaintext, requiring only scalar multiplication. 3) The Fusion Layer efficiently folds pooling scaling into the activation function.

As illustrated in Table 1, the runtime overhead is dominated by ciphertext-ciphertext multiplications. Standard architectures incur significant depth costs during pooling and activation steps. The proposed fusion layer mitigates this by integrating the pooling scaling factor directly into the polynomial activation. This optimization avoids an independent depth-consuming pooling step, effectively concentrating the computation into fewer high-cost operations. This reduction allows APKOS to support deeper architectures within the same encryption parameter constraints compared to baseline methods.

6.4 Running Time Overhead Investigation with and without Bootstrapping

To investigate the practical implications of bootstrapping in privacy-preserving KWS, we analyze its impact on computational overhead. In the CKKS scheme, the ciphertext level decreases by 1 after each homomorphic multiplication. When the level reaches 0, bootstrapping is required to refresh the ciphertext and enable further homomorphic operations. For example, bootstrapping is a necessary operation when using minimax composite polynomials to approximate ReLU (e.g., PReLU_{MC}) in existing privacy-preserving CNN inference schemes [34]. However, bootstrapping typically introduces significant computational overhead.

In contrast, the proposed PReLU_R does not require bootstrapping. We investigate the runtime implications by comparing PReLU_{MC} and PReLU_R . We follow the bootstrapping setup in [34] for implementing ResNet20 inference over HE ciphertexts. In this setting, PReLU_{MC} provides a 10-bit precision approximation and requires a multiplicative depth of 10. Since there are multiple ReLU activation functions in the original ResNet20 [34], multiple bootstrapping operations are required.

The multiplicative depth and runtime overhead of the original ResNet20 (with bootstrapping) and the proposed APKOS (without bootstrapping) are shown in Figures 6(a) and 6(b), respectively. We observe that bootstrapping with PReLU_{MC} in ResNet20 accounts for 97% of the total running time, which constitutes a significant overhead. In contrast, our APKOS, which avoids bootstrapping, incurs substantially less runtime overhead. Therefore, using the proposed PReLU_R significantly improves computational efficiency compared to PReLU_{MC} . Specifically,

compared to our APKOS, the original ResNet20 with PReLU_{MC} introduces an additional 5625.24 seconds of runtime overhead, accounting for 81.6% of our total inference time.

7 Experimental Results and Analysis

The experiments were conducted on a 64-bit Ubuntu 18.04.5 LTS PC equipped with an Intel Xeon Gold 6145 CPU @ 2.00GHz and 125GB RAM. Our APKOS was implemented in C++, utilizing the HEAAN library [31], based on the CKKS scheme. We configured the implementation with a ciphertext modulus of $Q = 2^{800}$ and a ring dimension of $N = 2^{11}$. Each ciphertext was designed to hold 2^{10} plaintext slots.

Our experiments utilized the widely used Google Speech Commands (GSC) dataset, which includes two versions: v1 and v2 [56]. These datasets consist of 1-second audio snippets recorded at a 16 kHz sampling rate, featuring short keyword utterances in natural environments. Specifically, v1 contains 65,000 snippets of 30 different words, while v2 includes 105,000 snippets of 35 different words. To align the dataset with the requirements of our model, we resampled the recordings to an 8000 Hz sampling rate. This adaptation preserved the essential characteristics of the data while ensuring compatibility with our model's input specifications. To partially reflect real-world acoustic variations, we additionally evaluated the model's robustness under additive noise (30/35/40 dB).

7.1 Execution Time and Cryptographic Modulus Analysis

The most closely related work, CryptCNet [60], employs complex-valued CNNs to achieve state-of-the-art performance in privacy-preserving KWS. In this sub-section, we compare our APKOS with CryptCNet using two key metrics: ciphertext modulus consumption and execution time. To further evaluate the effectiveness of the proposed fusion layer, we also include a variant of our model without the fusion layer. All three models are configured with the same cryptographic parameters to ensure a fair comparison.

Fig. 7 illustrates the ciphertext modulus consumed by the three models. The results show that APKOS consumes significantly less ciphertext modulus than CryptCNet. Moreover, the inclusion of the fusion layer in APKOS also effectively reduces the number of required scalar multiplications. The reduction in ciphertext modulus not only improves the numerical precision of the final KWS results, but also allows subsequent applications—such as automatic speech recognition and semantic inference—to operate on the processed output.

As for the running time, compared to CryptCNet, our approach achieves a shorter execution time while maintaining the same network depth. Specifically, if we do not use the fusion layer in APKOS, the total runtime is 6854.24 seconds. In contrast, the use of the fusion layer in APKOS reduces the runtime by approximately 8%, down to 6487.27 seconds. Additionally, our approach leverages the SIMD technique for batch processing of KWS tasks. The amortized inference time per audio input is only 6.3 seconds, which is 41% of the 16.4 seconds required by CryptCNet. These results demonstrate the efficiency advantage of our proposed method. To be clear, the 6.3 seconds is the batch processing amortization value via SIMD, which allows the cloud to process large batches of ciphertext inputs concurrently. While users may still experience non-trivial delay, the provider-side throughput and hardware cost can be controlled and scaled more effectively.

7.2 Comparison with State-of-the-Art Solutions

We compare the performance of our APKOS system with several state-of-the-art solutions for privacy-preserving KWS, including CryptCNet [60], CNN-B [28], HCNN [2], HEMET [39], ResNet20 [34], and Plain-18 [33]. The original ResNet20 and Plain-18 schemes rely on bootstrapping and thus cannot be realized solely by LHE. To adapt them to our bootstrapping-free setting, we use their modified versions TResNet and TPlain[60].

7.2.1 Accuracy. We show the accuracy of all the compared solutions in Table 2. CNN-B and HCNN underperformed because they achieve accuracies below 50%. The accuracy of TResNet and TPlain did not exceed 70%.

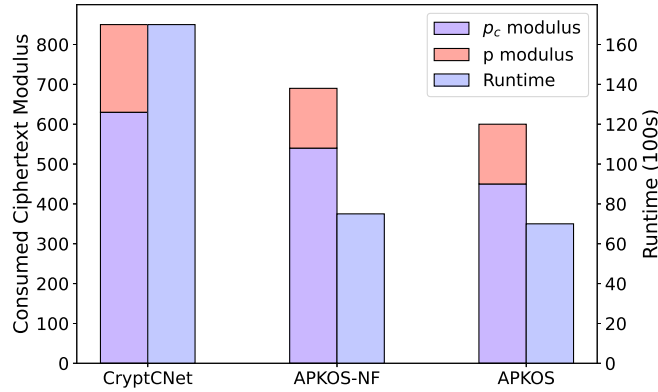


Fig. 7. Consumed ciphertext modulus and running time of different schemes. p_c denotes scalar multiplications, while p denotes non-scalar multiplications.

Since these two schemes were originally designed for image data characterized by sparse connectivity, they are less suitable for time-sequential audio data.

Among the existing schemes, CryptCNet achieved the second-best results. But they still fell short of 75% accuracy on both datasets. In contrast, our APKOS achieved over 80% accuracy, reaching 80.1% on GSC v2 and 82.1% on GSC v1, significantly outperforming all compared solutions.

Implication in practical CVAs. It is worth noting that APKOS targets the *second-stage* (cloud-side) verification in the widely used two-stage KWS pipeline. The first-stage on-device detector is optimized for high recall, while the cloud verifier mainly filters false triggers to prevent unnecessary uploading/processing. Therefore, the reported encrypted-domain accuracy should be interpreted as the effectiveness of the privacy-preserving cloud verification step, rather than the end-to-end wake-word experience of a production voice assistant.

7.2.2 Robustness. We evaluated the robustness of the solutions under noisy conditions by injecting Gaussian noise at signal-to-noise ratios (SNRs) of 40 dB, 35 dB, and 30 dB. The results are shown in Table 2. All schemes experienced accuracy degradation under noise. However, even at 30 dB, our APKOS maintained an accuracy of 77.3%, which is 8 percentage points higher than the second-best solution, CryptCNet.

7.2.3 F1 Score, Recall, and Precision. We further assessed the classification performance of all solutions using F1 score, recall, and precision. As shown in Table 2, our APKOS consistently outperformed other schemes across all three metrics.

7.2.4 ROC Curve. The Receiver Operating Characteristic (ROC) curve and the corresponding Area Under the Curve (AUC) measure a scheme's discrimination capability. Fig. 8 shows the ROC curves of all compared solutions on GSC v1 and v2.

Although CryptCNet demonstrated strong accuracy, its AUC-ROC was lower than those of TResNet, TPlain, and HEMET. Moreover, when the false positive rate exceeds 0.5, CryptCNet's true positive rate drops below that of CNN-B and HCNN, suggesting unstable behavior under lenient thresholds. In contrast, our APKOS achieved the most favorable ROC characteristics, with the sharpest curves and highest AUCs across both datasets.

7.2.5 PR Curve. The Precision-Recall (PR) curve offers further insight into the precision-recall balance of each solution. Fig. 9 presents the PR curves for all compared solutions.

Table 2. Performance comparison of accuracy, robustness, F1 score, recall, and precision.

Datasets	Method	Accuracy	SNR(dB)			F1 Score	Recall	Precision
			30	35	40			
GSC v1	Proposed	82.1	79.2	81.1	81.9	81.9	81.8	82.1
	CryptCNet[60]	<u>71.5</u>	<u>69.3</u>	<u>70.9</u>	<u>71.8</u>	<u>71.8</u>	<u>71.9</u>	<u>71.9</u>
	CNN-B[28]	35.0	27.8	31.6	33.7	35.4	35.5	35.7
	HCNN[2]	43.0	37.3	41.5	42.6	43.0	43.1	46.7
	HEMET[39]	52.9	46.9	50.9	52.6	53.3	53.0	54.8
	TResNet[34]	63.1	58.6	61.7	62.8	63.4	63.4	<u>63.9</u>
	TPlain[33]	69.5	63.3	67.2	69.1	69.5	69.4	70.1
GSC v2	Proposed	80.1	77.3	79.3	80.0	79.1	78.9	79.5
	CryptCNet[60]	<u>73.7</u>	<u>69.2</u>	<u>72.4</u>	<u>73.4</u>	<u>72.6</u>	<u>72.1</u>	<u>73.7</u>
	CNN-B[28]	37.8	28.9	32.8	36.2	36.3	36.5	36.4
	HCNN[2]	44.2	35.8	40.7	43.3	42.4	42.9	44.8
	HEMET[39]	53.3	48.3	51.9	52.9	52.0	51.9	52.8
	TResNet[34]	65.0	57.1	61.9	64.3	63.9	62.7	66.0
	TPlain[33]	67.4	60.3	64.4	66.7	65.7	65.5	66.3

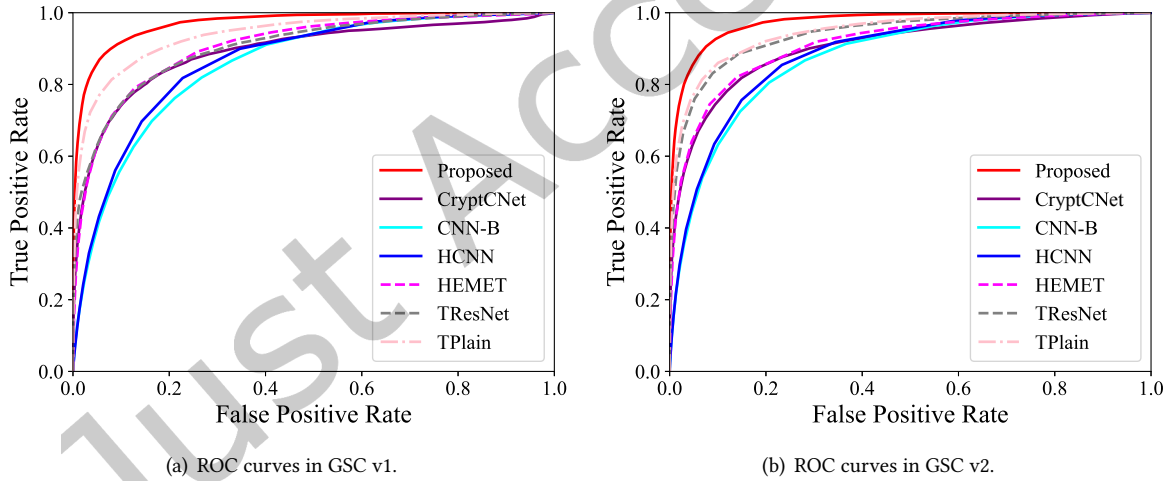


Fig. 8. Comparison of ROC curves for privacy-preserving KWS across different datasets.

Several solutions demonstrate clear performance degradation. The precision of CNN-B and HCNN drops rapidly as recall increases, revealing their limitations. On GSC v1, HEMET shows a sharp decline in precision when recall approaches zero. In contrast, APKOS maintains a convex PR curve with the highest AUC-PR values on both datasets. Therefore, our APKOS achieves the superior balance between precision and recall.

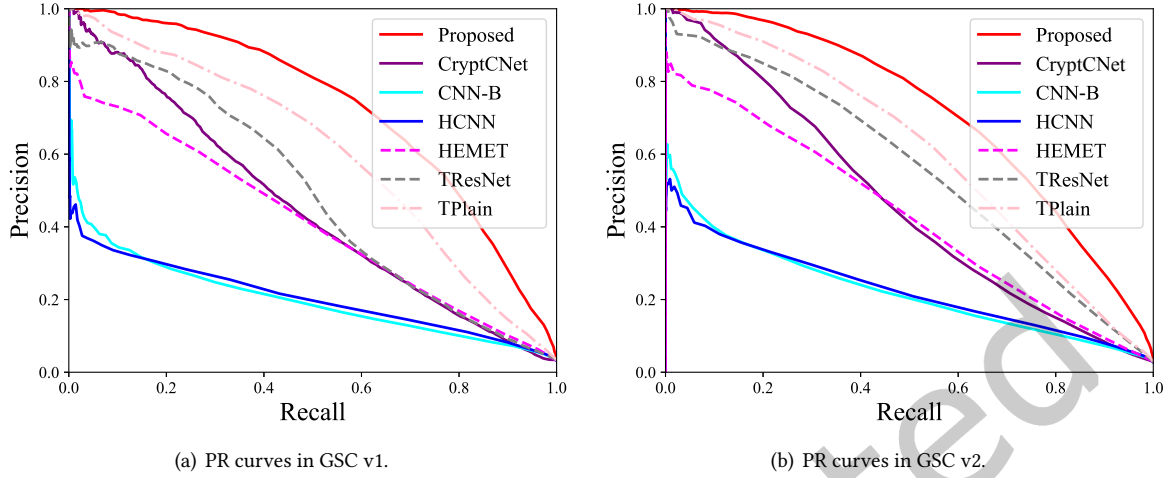


Fig. 9. Comparison of PR curves for privacy-preserving KWS across different datasets.

Table 3. Ablation study of activation functions.

	Proposed	CryptCNet[60]
PReLU _R	80.1	68.4
PReLU _C	77.6	73.7

7.3 Ablation Study

To investigate the influence of activation functions and kernel size on KWS performance, we conducted a series of ablation studies.

7.3.1 Activation Function. We compare two polynomial approximation functions in our ablation study. The activation function adopted by the most related secure KWS solution, CryptCNet [60], is PReLU_C. PReLU_C was reported to yield strong performance. However, PReLU_C may not be optimal for real-valued CNNs used in privacy-preserving KWS.

To validate this, we integrated both PReLU_C and our proposed PReLU_R into APKOS and CryptCNet, and evaluated their accuracy. The results in Table 3 show that PReLU_C performs slightly better within the original CryptCNet architecture. Applying PReLU_C to our APKOS also leads to higher accuracy than the original CryptCNet. However, it causes instability and convergence issues during training. In contrast, PReLU_R yields stable training behavior in APKOS, and achieves even higher accuracy than CryptCNet. These findings demonstrate the effectiveness of the proposed PReLU_R for our privacy-preserving KWS scheme.

7.3.2 Network Kernel Size. We further explore the impact of kernel size on KWS performance. Starting with a baseline model using small convolutional kernels, we gradually applied kernel expansion across the network. Fig. 10 illustrates this process. The initial expansion improved accuracy to 78.4%. However, further increases in kernel size resulted in performance degradation, even falling below the baseline. This suggests that global kernel enlargement across the network does not always yield performance gains.

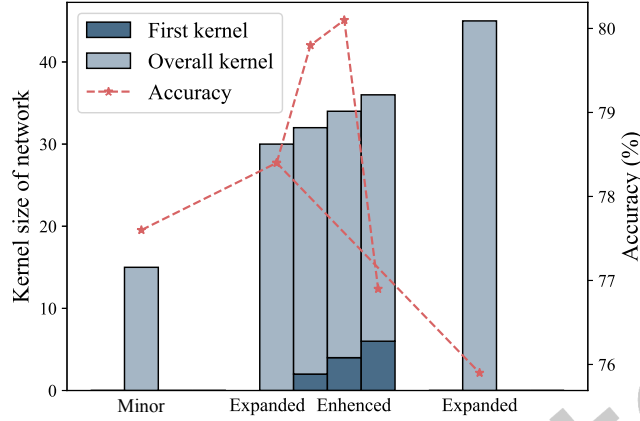


Fig. 10. Progressive effects of kernel expansion strategy on KWS performance. “Expanded” indicates uniform kernel enlargement across all layers; “Enhanced” denotes additional increase in the first-layer kernel size.

Table 4. Layer-wise kernel expansion and step-by-step performance comparison for KWS. “MiniConv” serves as the baseline model. “ExpandedConv” applies global kernel expansion. “EnhancedConv” further increases kernel sizes in critical layers.

Layer	MiniConv	ExpandedConv	EnhancedConv	Proposed
1 st	7	14	16	20
2 nd	7	16	16	16
3 rd	3	15	15	15
4 th	7	14	14	14
5 th	3	7	7	7
6 th	3	15	15	15
7 th	3	7	7	7
Accuracy(%)	77.6	78.4	79.8	80.1

We then focused on the first convolutional layer, which plays a critical role in capturing low-level temporal features. As shown in Table 4, after performing a global expansion, increasing the kernel size of the first convolutional layer by 2 leads to a notable performance improvement, while a further increase by 4 results in the best performance. Excessive enlargement beyond this point led to a sharp performance drop. The network structure used in our proposed APKOS is derived by applying a global kernel expansion along with a 6-unit increase (achieved through two successive expansions of +2 and +4) in the kernel size of the first convolutional layer.

7.3.3 Trade-off Between Accuracy and Efficiency. While enlarging convolutional kernels can enhance accuracy, it also increases runtime. We analyze the trade-off between accuracy and efficiency in Fig. 11. Our APKOS achieves significantly better performance while keeping the additional time overhead within acceptable limits.

We also evaluated how kernel expansion impacts other metrics. As shown in Table 5, our KWS system exhibits gradual improvements in F1 score, recall, and precision at each expansion step.

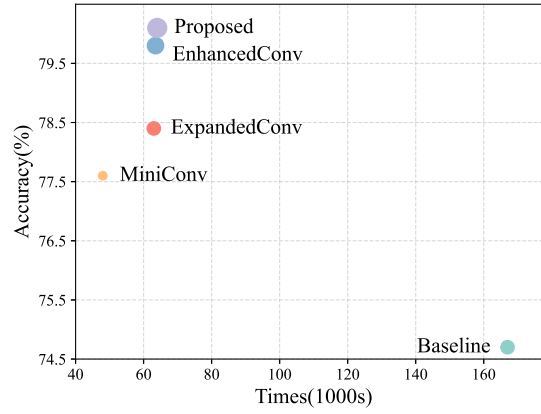


Fig. 11. Accuracy versus runtime under kernel expansion. Point area indicates convolution width; larger area means larger width. “Baseline” refers to the most related prior solution [60].

Table 5. Impact of kernel expansion on F1 score, recall, and precision.

Method	Accuracy	F1-score	Recall	Precision
Proposed	80.1	79.1	78.9	79.5
EnhancedConv	<u>79.8</u>	<u>79.0</u>	<u>78.8</u>	<u>79.4</u>
ExpandedConv	78.4	77.6	77.3	78.0
MiniConv	77.6	76.6	76.7	76.8

Table 6. Effect of applying the kernel expansion strategy across different privacy-preserving KWS schemes. “Original” denotes each method’s default configuration, “Expanded” indicates a uniform increase in kernel size across all layers, and “Enhanced” refers to further enlargement of the first-layer kernel on top of the expanded setting.

Method	Original	Expanded	Enhanced
Proposed	77.6	<u>78.4</u>	80.1
CryptCNet[60]	73.7	<u>75.4</u>	75.6
CNN-B[28]	37.8	<u>54.2</u>	56.0
HCNN[2]	44.2	<u>59.5</u>	60.6
HEMET[39]	53.3	<u>56.9</u>	58.6
TResNet[34]	65.0	<u>72.4</u>	72.7
TPlain[33]	67.4	<u>73.2</u>	73.6

7.3.4 Applying Kernel Expansion to Existing Solutions. To evaluate the generality of the proposed kernel expansion strategy, we applied it to all baseline solutions for privacy-preserving KWS. The results are presented in Table 6. Across all evaluated solutions, the kernel expansion strategy led to progressive accuracy improvements, demonstrating its broad applicability and effectiveness in the context of privacy-preserving KWS.

Table 7. Time consumption breakdown of the end-to-end process.

Party	Metric	Measurement	Notes
Client(per ciphertext)	Device-side encryption time	10.8 ms	encoding and encryption
	Serialization time	6.1 ms	preparing bytes for transmission
	Network transmission time	26 ms	from client to server over our real network testbed
	Total overhead	42.9 ms	--
Server	Secure inference	6.3 s	amortized per audio

7.4 Overheads Evaluation: End-to-End Time Consumption Breakdown

While server-side inference throughput is critical, a holistic evaluation must also account for end-to-end latency, including client-side processing and data transmission. We utilized an edge device (Intel Core i5, 4GB RAM, Ubuntu Linux) connected via a 100 Mbps network. In this environment, the client performed audio encoding, CKKS encryption, and serialization for a single 1-second voice command (8 kHz).

Combining all stages, Table 7 reports the end-to-end time consumption of the process. The results show that client-side computation is highly efficient, requiring only 16.9 ms (10.8 ms for encryption and 6.1 ms for serialization). Transmission to the cloud adds 26.0 ms, yielding a total non-server overhead of 42.9 ms. This performance demonstrates that the cryptographic burden is negligible for such resource-constrained edge devices, and the ciphertext payload fits comfortably within consumer broadband constraints. Furthermore, this overhead (≈ 43 ms) is orders of magnitude lower than the typical 1–2 second response latency of commercial voice assistants. These findings confirm that the client-side and network components are not system bottlenecks, leaving server computation as the dominant cost in our end-to-end pipeline. However, we emphasize, consistent with Section 7.1, that the server encrypted inference is a *throughput-oriented amortized* cost rather than a single-query latency, the delay is acceptable.

Table 8. Performance Comparison: APKOS vs. HE-Adapted Plaintext Models. “HE-Adapted Acc” denotes performance after modifying the architecture for HE compatibility. Note the instability of standard models.

Model	Params	Plaintext Acc	Adaptation v1 Acc	Adaptation v2 Acc	Outcome Status
QKWS [9]	3.2M	96.8%	NaN	77.5%	Significant Degradation
MatchboxNet [41]	80K	93.1%	NaN	NaN	Failed to Converge
BCResNet [32]	360K	93.5%	NaN	75.8%	Degradation
AttentionRNN [14]	180K	88.4%	NaN	67.3%	Unsuitable
APKOS (Ours)	360K	–	80.1%	80.1%	Optimal Stability

7.5 The Price of Privacy: Comparison with Adapted Plaintext Representative Models

To rigorously quantify the *Price of Privacy* and validate the necessity of our HE-specific design, we attempted to adapt four representative plaintext KWS models for the encrypted domain: QKWS [9], MatchboxNet [41], BCResNet [32], and AttentionRNN [14]. Since standard plaintext architectures rely on operations incompatible with CKKS (e.g., Batch Normalization, ReLU, Sigmoid), they cannot be deployed directly. We performed two rounds of rigorous HE-adaptation protocol: (1) Adaptation v1 (Naive Adaptation): Simply removed all Batch Normalization layers and replaced activation functions with polynomial approximations. As a result, all models failed to train due to gradient explosion (NaN). (2) Adaptation v2 (Optimized Adaptation): Based on v1, we further removed convolutional layers that only increase depth but do not change tensor dimensions, and carefully tuned

initialization. Table 8 presents the performance of these adapted models compared to their original plaintext accuracy and our proposed APKOS.

The results reveal a critical architectural insight: *Plaintext KWS architectures are extremely fragile under HE constraints*. Complex models like MatchboxNet failed to converge (diverging to NaN) due to the unbounded nature of polynomial activations (x^2, x^3), which cause gradient explosion in deep networks when stripped of Batch Normalization. Even models that converged (e.g., QKWS) suffered massive accuracy drops (>19%).

In stark contrast, APKOS is explicitly engineered for the HE environment. Our usage of the fusion Layer and specifically calibrated polynomial activations ensures numerical stability, allowing APKOS to achieve 80.1% accuracy—surpassing all adapted representative baselines. This confirms that privacy-preserving KWS requires specialized architectural design rather than the naive adaptation of existing plaintext models.

8 Limitations

Although APKOS achieves strong performance under the evaluated settings, several limitations should be noted. First, our experiments are conducted on benchmark datasets under relatively controlled conditions, and we have not yet systematically evaluated the framework in more diverse real-world scenarios, such as non-English speech, speakers with different accents, or overlapping multi-speaker voices. Therefore, the current results should be interpreted within the scope of the evaluated datasets and settings. Exploring these more challenging speech conditions and studying single-query deployment latency in greater detail will be important directions for future work.

9 Conclusion

In this paper, we proposed APKOS, an accurate and privacy-preserving KWS framework designed for CVA systems. Recognizing the privacy challenges inherent in CVA operations, we focused on protecting speech data during the cloud-stage KWS verification, where the most significant privacy risks occur. APKOS ensures semantic security for audio data while maintaining the full functionality of KWS through the use of deep learning techniques. To address the low KWS performance typically observed over homomorphically encrypted data in existing schemes, we designed a HE-friendly CNN architecture specifically tailored to the KWS task. We introduced a new polynomial activation function, proposed a novel fusion layer, and adopted larger kernel sizes to improve KWS performance while significantly reducing the computational overhead. Extensive experiments demonstrate that APKOS outperforms state-of-the-art secure KWS solutions in terms of efficiency, accuracy, robustness, ROC curve, and F1 score. By ensuring semantic security throughout the cloud-based processing stage, APKOS effectively mitigates privacy risks associated with voice data analysis.

The proposed APKOS framework is particularly well-suited as a secure second-stage verification layer for cloud voice assistants, offering a vital privacy safety net against false triggers. While current performance reflects a trade-off between strict cryptographic security and recognition accuracy, our polynomial activation and fusion designs are generalizable techniques. In future work, we aim to extend these methods to large-vocabulary continuous speech recognition in the encrypted domain—a key challenge in the field. Furthermore, we plan to address real-time latency constraints by integrating our algorithmic optimizations with emerging hardware acceleration technologies (e.g., FPGAs or ASICs).

References

- [1] Shima Ahmed, Ilia Shumailov, Nicolas Papernot, and Kassem Fawaz. 2022. Towards more robust keyword spotting for voice assistants. In *Proceedings of the USENIX Security Symposium (USENIX Security)*. USENIX Association, 2655–2672.
- [2] Ahmad Al Badawi, Chao Jin, Jie Lin, Chan Fook Mun, Sim Jun Jie, Benjamin Hong Meng Tan, Xiao Nan, Khin Mi Mi Aung, and Vijay Ramaseshan Chandrasekhar. 2021. Towards the AlexNet Moment for Homomorphic Encryption: HCNN, the First Homomorphic CNN on Encrypted Data With GPUs. *IEEE Transactions on Emerging Topics in Computing* 9, 3 (2021), 1330–1343.

- [3] Axel Berg, Mark O'Connor, and Miguel Tairum Cruz. 2021. Keyword Transformer: A Self-Attention Model for Keyword Spotting. arXiv:2104.00769 [eess.AS]
- [4] Song Bian, Dur E Shahwar Kundi, Kazuma Hirozawa, Weiqiang Liu, and Takashi Sato. 2021. APAS: Application-specific accelerators for RLWE-based homomorphic linear transformations. *IEEE Transactions on Information Forensics and Security* 16 (2021), 4663–4678.
- [5] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. 2014. (Leveled) Fully Homomorphic Encryption without Bootstrapping. *ACM Trans. Comput. Theory* 6, 3 (2014), 13:1–13:36.
- [6] Zhuojing Cai, Yuhua Ma, and Feng Lu. 2024. Robust Dual-Modal Speech Keyword Spotting for XR Headsets. *IEEE Transactions on Visualization and Computer Graphics* 30, 5 (2024), 2507–2516.
- [7] Nicholas Carlini and David Wagner. 2018. Audio adversarial examples: Targeted attacks on speech-to-text. In *Proceedings of the IEEE Security and Privacy Workshops (SPW)*. IEEE, 1–7.
- [8] Hervé Chabanne, Amaury de Wargny, Jonathan Milgram, Constance Morel, and Emmanuel Prouff. 2017. Privacy-Preserving Classification on Deep Neural Network. *IACR Cryptol. ePrint Arch.* 2017 (2017), 35.
- [9] Aryan Chaudhary and Vinayak Abrol. 2023. Towards on-device keyword spotting using low-footprint quaternion neural models. In *Proceedings of the IEEE Workshop on Applications of Signal Processing to Audio and Acoustics (WASPAA)*. IEEE, 1–5.
- [10] Hang Cheng, Huaxiong Wang, Ximeng Liu, Yan Fang, Meiqing Wang, and Xiaojun Zhang. 2021. Person Re-Identification over Encrypted Outsourced Surveillance Videos. *IEEE Transactions on Dependable and Secure Computing* 18, 3 (2021), 1456–1473.
- [11] J. H. Cheon, K. Han, M. Kim A. Kim, and Y. Song. 2018. A full RNS variant of approximate homomorphic encryption. In *Proceedings of the International Conference on Selected Areas in Cryptography (SAC)*. Springer, 347–368.
- [12] Jung Hee Cheon, Minsik Kang, Taeseong Kim, Junyoung Jung, and Yongdong Yeo. 2025. Batch Inference on Deep Convolutional Neural Networks With Fully Homomorphic Encryption Using Channel-By-Channel Convolutions. *IEEE Transactions on Dependable and Secure Computing* 22, 2 (2025), 1674–1685.
- [13] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. 2017. Homomorphic encryption for arithmetic of approximate numbers. In *Proceedings of the International Conference on the Theory and Application of Cryptology and Information Security (ASIACRYPT)*. Springer, 409–437.
- [14] Douglas Coimbra De Andrade, Sabato Leo, Martin Loesener Da Silva Viana, and Christoph Bernkopf. 2018. A Neural Attention Model for Speech Command Recognition. arXiv:1808.08929 [eess.AS]
- [15] Piet De Vaere and Adrian Perrig. 2023. Hey Kimya, Is My Smart Speaker Spying on Me? Taking Control of Sensor Privacy Through Isolation and Amnesia. In *Proceedings of the USENIX Security Symposium (USENIX Security)*. USENIX Association, 2401–2418.
- [16] Kevin Ding, Martin Zong, Jiakui Li, and Baoxiang Li. 2022. LETR: A Lightweight and Efficient Transformer for Keyword Spotting. In *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 7987–7991.
- [17] Xiaohan Ding, Xiangyu Zhang, Jungong Han, and Guiguang Ding. 2022. Scaling up your kernels to 31x31: Revisiting large kernel design in CNNs. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 11963–11975.
- [18] Daniel J. Dubois, Roman Kolcun, Anna Maria Mandalari, Muhammad Talha Paracha, David R. Choffnes, and Hamed Haddadi. 2020. When speakers are all ears: Characterizing misactivations of IoT smart speakers. *Proceedings on Privacy Enhancing Technologies* 2020 (2020), 255 – 276.
- [19] Jide S Edu, Jose M Such, and Guillermo Suarez-Tangil. 2020. Smart home personal assistants: a security and privacy review. *ACM Computing Surveys (CSUR)* 53, 6 (2020), 1–36.
- [20] Junfeng Fan and Frederik Vercauteren. 2012. Somewhat Practical Fully Homomorphic Encryption. *IACR Cryptol. ePrint Arch.* 2012 (2012), 144.
- [21] Yahoo Finance. 2023. Voice Assistant Application Market Set to Grow. <https://finance.yahoo.com/news/voice-assistant-application-market-set-215000277.html>
- [22] Ran Gilad-Bachrach, Nathan Dowlin, Kim Laine, Kristin Lauter, Michael Naehrig, and John Wernsing. 2016. CryptoNets: Applying neural networks to encrypted data with high throughput and accuracy. In *Proceedings of the International Conference on Machine Learning (ICML)*. PMLR, 201–210.
- [23] Heitor R Guimarães, Arthur Pimentel, Anderson Avila, and Tiago H Falk. 2024. VIC-KD: Variance-Invariance-Covariance Knowledge Distillation to Make Keyword Spotting More Robust Against Adversarial Attacks. In *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 12196–12200.
- [24] S. Hendery. 2023. Amazon to pay 30.8M for Alexa and Ring privacy violations. <https://scmagazine.com/news/amazon-30-8m-alexar-ing-privacy-violations>
- [25] Ehsan Hesamifard, Hassan Takabi, and Mehdi Ghasemi. 2017. CryptoDL: Deep Neural Networks over Encrypted Data. arXiv:1711.05189 [cs.CR]
- [26] Yan Huang, Jonathan Katz, and David Evans. 2012. Quid-pro-quo-tocols: Strengthening semi-honest protocols with dual execution. In *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*. IEEE, 272–284.
- [27] Zhicong Huang, Wen-jie Lu, Cheng Hong, and Jiansheng Ding. 2022. Cheetah: Lean and fast secure two-party deep neural network inference. In *Proceedings of the USENIX Security Symposium (USENIX Security)*. USENIX Association, 809–826.

- [28] Takumi Ishiyama, Takuya Suzuki, and Hayato Yamana. 2020. Highly Accurate CNN Inference Using Approximate Activation Functions over Homomorphic Encryption. In *Proceedings of the IEEE International Conference on Big Data (Big Data)*. IEEE, 3989–3995.
- [29] Linzhi Jiang, Chunxiang Xu, Xiaofang Wang, Bo Luo, and Huaqun Wang. 2020. Secure outsourcing SIFT: Efficient and Privacy-Preserving Image Feature Extraction in the Encrypted Domain. *IEEE Transactions on Dependable and Secure Computing* 17, 1 (2020), 179–193.
- [30] Jae Hyung Ju, Jaiyoung Park, Jongmin Kim, Minsik Kang, Donghwan Kim, Jung Hee Cheon, and Jung Ho Ahn. 2024. NeuJeans: Private Neural Network Inference with Joint Optimization of Convolution and FHE Bootstrapping. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 4361–4375.
- [31] A. Kim. 2018. HEAAN. <https://github.com/kimandrik/HEAAN>
- [32] Byeonggeun Kim, Simyung Chang, Jinkyu Lee, and Dooyong Sung. 2021. Broadcasted Residual Learning for Efficient Keyword Spotting. arXiv:2106.04140 [eess.AS]
- [33] Dongwoo Kim and Cyril Guyot. 2023. Optimized Privacy-Preserving CNN Inference With Fully Homomorphic Encryption. *IEEE Transactions on Information Forensics and Security* 18 (2023), 2175–2187.
- [34] Eunsang Lee, Joon-Woo Lee, Junghyun Lee, Young-Sik Kim, Yongjune Kim, Jong-Seon No, and Woosuk Choi. 2022. Low-complexity deep convolutional neural networks on fully homomorphic encryption using multiplexed parallel convolutions. In *Proceedings of the International Conference on Machine Learning (ICML)*. PMLR, 12403–12422.
- [35] Eunsang Lee, Joon-Woo Lee, Jong-Seon No, and Young-Sik Kim. 2021. Minimax approximation of sign function by composite polynomial for homomorphic comparison. *IEEE Transactions on Dependable and Secure Computing* 19, 6 (2021), 3711–3727.
- [36] R. Lerman. 2021. Lawsuits say Siri and Google are listening, even when they’re not supposed to. <https://washingtonpost.com/technology/2021/09/02/apple-siri-lawsuit-privacy/>
- [37] Shaohua Li, Kaiping Xue, Bin Zhu, Chenkai Ding, Xindi Gao, David Wei, and Tao Wan. 2020. Falcon: A fourier transform based approach for fast and secure convolutional neural network predictions. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 8705–8714.
- [38] Yehuda Lindell. 2017. How to Simulate It – A Tutorial on the Simulation Proof Technique. In *Tutorials on the Foundations of Cryptography: Dedicated to Oded Goldreich*. Springer, 277–346.
- [39] Qian Lou and Lei Jiang. 2021. HEMET: A homomorphic-encryption-friendly privacy-preserving mobile neural network architecture. In *Proceedings of the International Conference on Machine Learning (ICML)*. PMLR, 7102–7110.
- [40] Vadim Lyubashevsky, Chris Peikert, and Oded Regev. 2010. On ideal lattices and learning with errors over rings. In *Proceedings of the Annual International Conference on the Theory and Applications of Cryptographic Techniques (EUROCRYPT)*. Springer, 1–23.
- [41] Somshubra Majumdar and Boris Ginsburg. 2020. MatchboxNet: 1D Time-Channel Separable Convolutional Neural Network Architecture for Speech Commands Recognition. arXiv:2004.08531 [eess.AS]
- [42] Lydia Manikonda, Aditya Deotale, and Subbarao Kambhampati. 2018. What’s up with privacy? User preferences and privacy concerns in intelligent personal assistants. In *Proceedings of the AAAI/ACM Conference on AI, Ethics, and Society (AI/ES)*. ACM, 229–235.
- [43] Nicole Meng, Dilara Keküllüoğlu, and Kami Vaniea. 2021. Owning and sharing: Privacy perceptions of smart speaker users. *Proceedings of the ACM on Human-Computer Interaction* 5, CSCW1, Article 45 (2021), 45:1–45:29 pages.
- [44] Aviv Navon, Aviv Shamsian, Neta Glazer, Gill Hetz, and Joseph Keshet. 2024. Open-Vocabulary Keyword-Spotting with Adaptive Instance Normalization. In *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 11656–11660.
- [45] Dianwen Ng, Yunqi Chen, Biao Tian, Qiang Fu, and Eng Siong Chng. 2022. Convmixer: Feature Interactive Convolution with Curriculum Learning for Small Footprint and Noisy Far-Field Keyword Spotting. In *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 3603–3607.
- [46] Dianwen Ng, Jin Hui Pang, Yang Xiao, Biao Tian, Qiang Fu, and Eng Siong Chng. 2022. Small Footprint Multi-channel ConvMixer for Keyword Spotting with Centroid Based Awareness. arXiv:2204.05445 [eess.AS]
- [47] Dianwen Ng, Ruixi Zhang, Jia Qi Yip, Chong Zhang, Yukun Ma, Trung Hieu Nguyen, Chongjia Ni, Eng Siong Chng, and Bin Ma. 2023. Contrastive speech mixup for low-resource keyword spotting. In *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 1–5.
- [48] David Peter, Wolfgang Roth, and Franz Pernkopf. 2022. End-to-end keyword spotting using neural architecture search and quantization. In *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 3423–3427.
- [49] Ran Ran, Xinwei Luo, Wei Wang, Tao Liu, Gang Quan, Xiaolin Xu, Caiwen Ding, and Wujie Wen. 2023. SpENCNN: orchestrating encoding and sparsity for fast homomorphically encrypted neural network inference. In *Proceedings of the International Conference on Machine Learning (ICML)*. PMLR, 28718–28728.
- [50] Oleg Rybakov, Natasha Kononenko, Niranjan Subrahmanya, Mirkó Visontai, and Stella Laurenzo. 2020. Streaming Keyword Spotting on Mobile Devices. arXiv:2005.06720 [eess.AS]
- [51] Lea Schönherr, Maximilian Golla, Thorsten Eisenhofer, Jan Wiele, Dorothea Kolossa, and Thorsten Holz. 2020. Unacceptable, Where Is My Privacy? Exploring Accidental Triggers of Smart Speakers. arXiv:2008.00508 [cs.CR]

- [52] Siddharth Sigtia, Rob Haynes, Hywel Richards, Erik Marchi, and John Bridle. 2018. Efficient Voice Trigger Detection for Low Resource Hardware.. In *Proceedings of the Annual Conference of the International Speech Communication Association (INTERSPEECH)*. ISCA, 2092–2096.
- [53] Amit Kumar Sikder, Hidayet Aksu, and A. Selcuk Uluagac. 2020. A Context-Aware Framework for Detecting Sensor-Based Threats on Smart Devices. *IEEE Transactions on Mobile Computing* 19, 2 (2020), 245–261.
- [54] Ming Sun, David Snyder, Yixin Gao, Varun Nagaraja, Mike Rodehorst, Sankaran Panchapagesan, Nikko Ström, Spyros Matsoukas, and Shiv Vitaladevuni. 2017. Compressed Time Delay Neural Network for Small-footprint Keyword Spotting. In *Proceedings of the Annual Conference of the International Speech Communication Association (INTERSPEECH)*. ISCA, 1473–1477.
- [55] Paola Vitolo, Rosalba Liguori, Luigi Di Benedetto, Alfredo Rubino, and Gian Domenico Licciardo. 2024. Automatic Audio Feature Extraction for Keyword Spotting. *IEEE Signal Processing Letters* 31 (2024), 161–165.
- [56] Pete Warden. 2017. Speech Commands: A Public Dataset for Single-Word Speech Recognition. http://download.tensorflow.org/data/speech_commands_v0.01.tar.gz.
- [57] J.G. Wilpon, L.G. Miller, and P. Modi. 1991. Improvements and applications for key word recognition using hidden Markov modeling techniques. In *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 309–312.
- [58] Kaiping Xue, Weikeng Chen, Wei Li, Jianan Hong, and Peilin Hong. 2018. Combining data owner-side and cloud-side access control for encrypted cloud storage. *IEEE Transactions on Information Forensics and Security* 13, 8 (2018), 2062–2074.
- [59] Mengjun Zeng and Nanfeng Xiao. 2019. Effective combination of DenseNet and BiLSTM for keyword spotting. *IEEE Access* 7 (2019), 10767–10775.
- [60] Peijia Zheng, Zhiwei Cai, Huicong Zeng, and Jiwu Huang. 2022. Keyword Spotting in the Homomorphic Encrypted Domain Using Deep Complex-Valued CNN. In *Proceedings of the ACM International Conference on Multimedia (MM)*. ACM, 1474–1483.
- [61] Man Zhou, Zhan Qin, Xiu Lin, Shengshan Hu, Qian Wang, and Kui Ren. 2019. Hidden voice commands: Attacks and defenses on the VCS of autonomous driving cars. *IEEE Wireless Communications* 26, 5 (2019), 128–133.

Received 12 June 2025; revised 10 June 2026; accepted 13 July 2026